



Module Code	:	CT069-3-3-DBS
Intake Code	:	APD3F2601CS(CYB)
Coursework Title	:	SMARTBANK MALAYSIA
Lecturer Name	:	DR. KULOTHUNKAN PALASUNDRAM
Hand out Date	:	23 RD JUNE 2026
Hand in Date	:	29 TH AUGUST 2026
Lab No.	:	LAB-8
Group No.	:	G13
Group Leader	:	HEMANTHRAJAH A/L N KRISHNARAJAH

Student ID	Student Name
TP071194	ADAM JEROME JOSEPH
TP070385	GURSHAREENJIT KAUR A/P KARAMJIT SINGH
TP070844	HEMANTHRAJAH A/L KRISHNARAJAH
TP070996	SAILESS RAJ A/L RAJENDRAN

Table of Contents

Executive Summary	4
1. Introduction.....	5
1.1 Workload Matrix	6
2. Assignment Requirements and Security Objectives.....	7
3. Security Evaluation of the Initial Database	8
4. Secure Database Design and ERD	9
4.1 Main entities and relationships.....	10
4.2 Data Dictionary and Data Classification.....	11
4.3 Database Hardening and Additional Integrity Controls.....	13
5. Confidentiality Implementation and Evidence	15
5.1 Authentication, users, roles and permission model.....	16
5.2 Authorization Matrix, RBAC and Security Metadata	17
5.3 Row-Level Security	18
5.4 Secure Views	18
5.5 DBA Access Test.....	19
5.6 Bank Manager Access and Staff-Management Test	20
5.7 Bank Officer Access and Customer-Account Management Test	21
5.8 Customer 01 Access and Transaction Test.....	22
5.9 Customer 02 Isolation Test.....	23
5.10 Encryption Design and Ciphertext Storage	24
5.11 Dynamic Masking, Static Masking, Hashing and Anonymization	25
5.12 Automatic Authorised Decryption	25
5.13 Additional Security Auditor Role	26
6. Integrity Implementation and Evidence	27
6.1 Controlled DML and separation of duties.....	27
6.2 Role-Specific Modification Responsibilities	27
6.3 Valid and Invalid Transaction Controls.....	28
6.4 Idempotency and Financial Transaction Traceability	28
6.5 Audit Strategy and Audit Matrix	28
6.6 Data Manipulation Language (DML) Audit.....	29
6.7 Business-Action, DDL and SQL Server Audit Evidence	29
6.8 Permission and Role-Change Audit	30

6.9 Login, Logout and Failed-Login Audit.....	31
6.10 Temporal History and Concurrency Controls	31
7. Availability, Backup and Recovery.....	32
7.1 SQL Server/Agent status and automated backup jobs.....	32
7.2 Cross-instance encrypted restore preparation	34
7.3 Encrypted backup restored on the second SQL Server instance	37
7.4 Point-in-time recovery (PITR) test	40
8. Consolidated Test Cases.....	48
9. Assessment Rubric Coverage Matrix	51
10. Demonstration Readiness Mapping.....	52
11. Limitations and Recommended Improvements	53
12. Conclusion	54
References	55
Appendix A - Evidence Figure Index.....	56
Appendix B - SQL Excerpts Visible in Evidence	58
B.1 Deliberate PITR deletion	58
B.2 Manual post-deletion log backup	58
B.3 Cross-instance restore file visibility check	58
B.4 Final recovered row verification	58

Executive Summary

This report shows the security evaluation, redesign, implementation and testing of SmartBankDB for SmartBank Malaysia. The solution is built around Confidentiality, Integrity and Availability (CIA) and directly tackles the requirements of the assignment for the authenticated access, role based permissions, controlled modification, auditing, automated backup/recoverability. The deployed design includes SQL Server authentication, user-defined roles, explicit GRANT/DENY, Row-Level Security (RLS), secure views, stored procedures, Dynamic Data Masking (DDM), AES-256 column-level encryption, salted SHA2-256 hashing, sensitivity classification, system-versioned history, SQL Server Audit, custom audit tables, SQL Server Agent jobs, AES-256 encrypted backups, and cross-instance restore and point-in-time recovery.

Evidence contains configuration and security behaviour in real. DBA and Bank Manager and Bank Officer sessions have different permissions; the Customer sessions are constrained to their own account and transaction rows; the Customer sessions have authorized values through controlled decryption; invalid transactions and operations are rejected; valid and invalid operations are auditable; three days of scheduled full, differential and transaction-log backups are successful; an encrypted backup is restored on a separate SQL Server instance; and a STOPAT point-in-time restore is used successfully to recover a deliberately deleted transaction. Further defence-in-depth measures are added – such as an independent Security Auditor role, a queryable data dictionary, audit and authorisation matrices, temporal history, idempotency keys, rowversion columns and masked analytical snapshots – to the minimum requirements of the case study.

1. Introduction

SmartBank Malaysia - A Digital Banking Case Study is a case study that uses a centralised Microsoft SQL Server database for staff, customers, accounts and financial transactions. The original developer schema should be reviewed and redesigned to ensure that at a database level, controls are established to ensure the confidentiality, integrity and availability of the database, while maintaining the legitimate functions of the DBA, Bank Manager, Bank Officer and Customer users. The redesign thus sees the database itself as an active security perimeter in addition to the reliance on application code.

A defence in depth approach is used in the re design. Authentication is for the connecting principal, role-based permissions are for the operations that can be performed, secure views and RLS are for data exposure, DDM is for unnecessary contact data display, AES-256 encryption is for data at rest, stored procedures are for business rules and ownership checks, and multiple audit mechanisms are there for accountability. The FULL recovery model, automated full/differential/log backup, availability and checksum verification, backup encryption, and cross-instance restore and point-in-time recovery are supported. The emphasis of the course is on least privilege, separation of duties, layered protection, auditing and recoverability, which are all reflected in the following controls (Microsoft, 2025a; Microsoft, 2026a).

The report is based on evidence. Every major control is associated to the SQL objects that it has been implemented and, if present, to screenshots of the deployed SmartBankDB environment. The implementation SQL provided with the assignment is considered authoritative for the object names and security behaviour – screenshots are used to demonstrate that the controls are working as expected.

1.1 Workload Matrix

GROUP MEMBERS	PERCENTAGE (%)
ADAM JEROME JOSEPH (TP071194)	24
GURSHAREENJIT KAUR A/P KARAMJIT SINGH (TP070385)	24
HEMANTHRAJAH A/L KRISHNARAJAH (TP070844)	28
SAILESS RAJ A/L RAJENDRAN (TP070996)	24
Total	100

2. Assignment Requirements and Security Objectives

There are four types of operational users defined in the assignment: DBA, Bank Manager, Bank Officer and Customer. All users should be authenticated; authenticated users should only see staff names and contact numbers; each role should only be able to see a subset of staff, account, and transaction access as dictated by the case study; access to modify the database and the data within it should be controlled by each user's role; valid and invalid manipulations should be logged; and the system should be recoverable with an explicitly justified RPO and automated full, differential, and transaction-log backups. The live demo also needs to show multiple logins, role-based permissions, audit logging, three days of the successful scheduling of backups, encryption of restore to another SQL instance, point-in-time recovery after deletion, auditable permissions changes and auto-decryption for the appropriate authorised user.

CIA area	Required outcome	Implemented control
Confidentiality	Prevent unauthorized reading while preserving required access	SQL logins/users, custom roles, GRANT/DENY, stored procedures, RLS, DDM, encryption, hashing, sensitivity classification
Integrity	Prevent unauthorized/invalid changes and record all manipulation	Stored procedures, direct DML denial, transaction validation, controlled role permissions, application audit, DDL audit, SQL Server Audit
Availability	Keep data accessible and recoverable	FULL recovery model, automated full/differential/log backups, backup verification, AES-256 encryption, cross-instance restore, PITR

3. Security Evaluation of the Initial Database

The assignment contains only four basic tables: Staff, Customer, Account and TransactionRecord, covered in Appendix 1, with little integrity and no specific security design. The schema is adequate for the basic storage of banking data, but doesn't in itself implement the CIA requirements. The key areas of strength and weakness are:

- There is no authentication or role model set in the database script, therefore the needed separation between DBA, manager, officer, and customer is not enforced.
- Sensitive data like staff salary, customer IC number, customer address, customer account balance or transaction amount is stored as a normal column without any encryption or masking method.
- Row-Level Security is not defined, therefore a customer level restriction such as “own accounts only” would be fully reliant on application logic.
- No explicit foreign key restrictions are defined between Customer, Account, and TransactionRecord, which increases the danger of orphaned or inconsistent entries.
- No stored processes or restricted interfaces for deposits, withdrawals and transfers, thus direct DML might bypass business regulations.
- No audit method defined for DDL, DML, successful activities or failed/invalid actions.
- There is no automated backup schedule and no backup encryption/verification or point-in-time recovery mechanism.

The redesign that was implemented includes complementary controls to counteract these weaknesses, but not one single mechanism. permissions / views minimize exposure, database tier ownership via RLS, sensitive identifiers are encrypted / hashed, business validation centralized via procedures, audit layers ensure accountability, and a well-tested backup/recovery chain provides availability. DDM is not meant to be a security perimeter in and of itself, but instead is used as a presentation control, in conjunction with permissions, auditing, and encryption (Microsoft, 2026c).

4. Secure Database Design and ERD

The security architecture that was implemented in SmartBankDB is shown in Figure 1: The key elements of a banking system (core banking entities) are presented along with identity mapping, authorisation documentation, temporal history, audit evidence and masked reporting structures, which are all essential to the deployed security model.

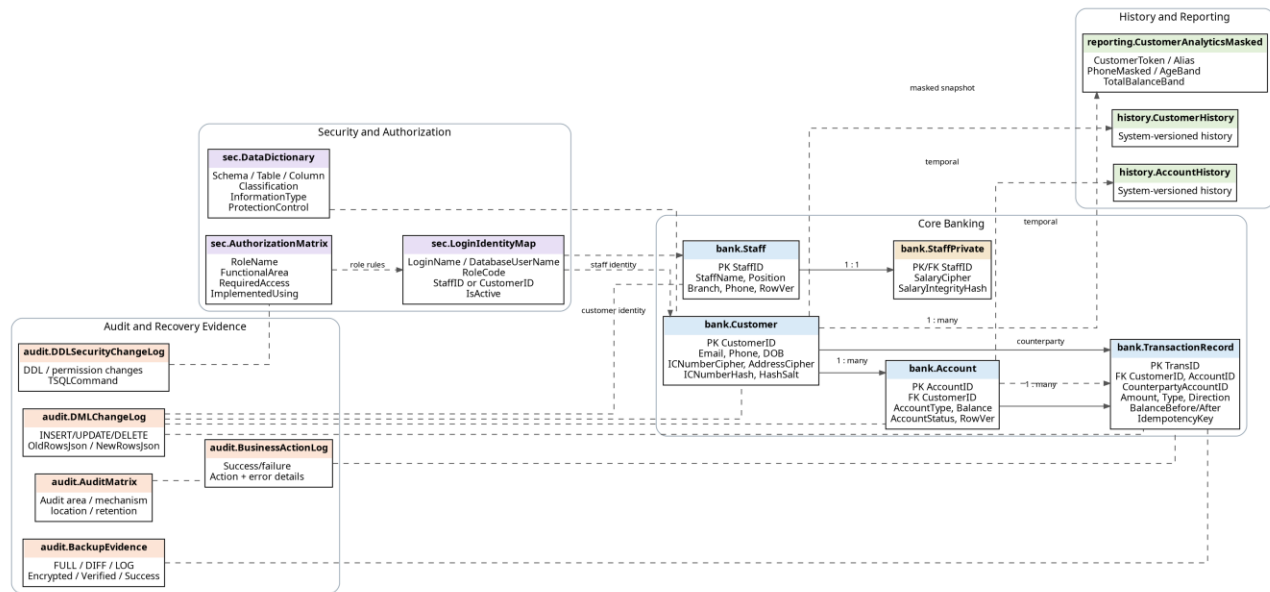


Figure 1. SmartBankDB secure logical entity-relationship diagram.

The primary business relationships are Customer one-to-many Account, and Account one-to-many TransactionRecord, where Customer is also noted on each transaction for purpose of ownership and RLS calculations. The payroll data is divided into banks. StaffPrivate restricts access to private compensation information; sec.LoginIdentityMap associates authenticated logon database users with StaffID or CustomerID; audit tables contain business activity, row updates, DDL, security updates and backup results. System-versioned Customer and Account history tables preserve previous row versions, and reporting.CustomerAnalyticsMasked offers a low exposure analytics snapshot.

The diagram has been created from the final implementation scripts provided with the assignment so the names of the tables and schemas in this report are the same as the deployed objects in SQL Server.

4.1 Main entities and relationships

Entity	Purpose	Key security relevance
bank.Staff	Public/operational staff profile	Supports public staff directory while separating salary from common staff data.
bank.StaffPrivate	Private staff attributes such as encrypted salary	Restricts salary disclosure and enables authorized decryption.
bank.Customer	Customer identity/profile	Contains protected personal data including encrypted IC number and address.
bank.Account	Customer bank accounts and balances	RLS restricts rows to the owning customer for customer sessions.
bank.TransactionRecord	Deposits, withdrawals and transfer entries	Stores audit-friendly transaction metadata such as before/after balances, initiator and idempotency key.
sec.LoginIdentityMap	Maps authenticated database users to StaffID or CustomerID	Connects database authentication to business identity, role behaviour and RLS ownership.
audit.BusinessActionLog	Business/security action audit	Records successful and rejected operations with actor, action, result and error details.
audit.DDLSecurityChangeLog	DDL/security change audit	Records CREATE/ALTER/DROP and permission changes together with T-SQL command text.
audit.BackupEvidence	Backup run evidence	Records backup type, encryption, verification and SQL Agent job outcome.
reporting.CustomerAnalyticsMasked	Masked analytical projection	Supports reduced-exposure reporting and analytics.

audit.DMLChangeLog	Successful row-change audit	Captures INSERT/UPDATE/DELETE actions with before/after row images stored as JSON.
--------------------	--------------------------------	---

4.2 Data Dictionary and Data Classification

sec.DataDictionary is a queryable data dictionary. It indicates the purpose of each individual column deemed to be security relevant with four classifications – Public, Internal, Confidential and Private – and provides information type, business purpose and protection control applied. This explicitly meets the assignment requirement of having a data dictionary that has data classification available, and it is a repeatable source, for the Security Auditor to query during the demonstration.

Table	Column	Type	Class	Purpose	Protection control
bank.Staff	StaffID	char(6)	Internal	Employee identifier	PK; RBAC; not exposed publicly
bank.Staff	StaffName	nvarchar(100)	Public	Staff identity	Public directory view only
bank.Staff	Position	varchar(20)	Internal	Job role	CHECK + filtered views/procedures
bank.Staff	Branch	nvarchar(50)	Internal	Work location	RBAC + self-service procedure
bank.Staff	Phone	varchar(20)	Public	Staff contact	Public directory; base table restricted
bank.StaffPrivate	SalaryCipher	varbinary(256)	Private	Financial information	AES-256 authenticated ciphertext; authorised procedures
bank.Customer	CustomerID	char(6)	Internal	Customer identifier	PK + identity/RLS mapping
bank.Customer	CustomerName	nvarchar(100)	Internal	Customer servicing	RBAC/views; direct table access denied
bank.Customer	Email	varchar(254)	Confidential	Contact information	DDM; UNMASK only for Bank Officer

bank.Customer	Phone	varchar(20)	Confidential	Contact information	DDM; UNMASK only for Bank Officer
bank.Customer	DateOfBirth	date	Confidential	Identity verification	DDM + secure own-profile procedure
bank.Customer	ICNumberCipher	varbinary(256)	Private	National identifier	AES-256 + per-record authenticator
bank.Customer	AddressCipher	varbinary(512)	Private	Address/KYC	AES-256 + per-record authenticator
bank.Customer	ICNumberHash	varbinary(32)	Private	One-way duplicate/integrity support	Salted SHA2-256 hash
bank.Account	AccountID	char(10)	Confidential	Financial account identifier	RLS + views + FK + DML denial
bank.Account	CustomerID	char(6)	Confidential	Account ownership	FK + RLS predicate
bank.Account	Balance	decimal(15,2)	Private	Available balance	RLS + procedures + CHECK + encrypted backup
bank.TransactionRecord	TransID	bigint	Internal	Transaction identifier	Identity PK; audit traceability
bank.TransactionRecord	Amount	decimal(15,2)	Private	Financial transaction	RLS + CHECK + secure procedures
bank.TransactionRecord	TransactionType	varchar(12)	Confidential	Deposit/withdrawal/transfer	CHECK constraint
audit.BusinessActionLog	ErrorMessage	nvarchar(2048)	Confidential	Valid/invalid action evidence	Auditor read-only; encrypted backup
audit.DMLChangeLog	OldRowsJson	nvarchar(max)	Private	Before-image change history	Restricted auditor access + encrypted backup
audit.DDLSecurityChangeLog	TSQLCommand	nvarchar(max)	Confidential	DDL/permission evidence	DDL trigger + SQL Server Audit

SQL Server sensitivity classifications additionally mark SalaryCipher, ICNumberCipher, AddressCipher, Account.Balance and TransactionRecord.Amount as HIGH or CRITICAL Private data. Figure 3 shows these classifications in the deployed environment.

4.3 Database Hardening and Additional Integrity Controls

The final schema goes beyond the Appendix 1 design by adding relational, concurrency and historical controls that reduce opportunities for inconsistent or duplicate banking data. These controls are implemented directly in SQL Server and complement the role and procedure security described later.

Control	Implemented on	Security benefit
Foreign keys	StaffPrivate->Staff; Account->Customer; TransactionRecord->Customer/Account/counterparty	Prevents orphaned relationships and maintains referential integrity.
CHECK constraints	IDs, positions, phone length, customer/account status, balance ≥ 0 , amount > 0 , valid transaction type/direction	Rejects invalid values before they can become persistent data.
rowversion	Staff, StaffPrivate, Customer and Account	Supports optimistic-concurrency detection and evidence of row change.
System-versioned temporal tables	bank.Customer and bank.Account with history.CustomerHistory/history.AccountHistory	Preserves prior row versions for investigation and historical analysis.
Idempotency key	Unique filtered index on TransactionRecord.Idempotency Key	Prevents duplicate financial posting when a request is retried.
FULL recovery model	SmartBankDB	Enables transaction-log backup and point-in-time recovery.
PAGE_VERIFY CHECKSUM	SmartBankDB	Improves detection of page-level corruption.

READ_COMMITTED_SNAPSHOT ON	SmartBankDB	Reduces reader/writer blocking while maintaining transaction isolation semantics.
TRUSTWORTHY OFF	SmartBankDB	Reduces unnecessary cross-database trust and privilege-escalation surface.

5. Confidentiality Implementation and Evidence

Confidentiality is implemented through layered authentication, role-based authorisation and data-protection controls. SQL logins map to database users and user-defined roles; permissions are assigned to roles using explicit GRANT and DENY statements; RLS limits rows by authenticated identity; secure views expose only required columns; DDM reduces unnecessary display of contact data; and AES-256 encryption protects highly sensitive salary, IC-number and address values. This follows the least-privilege model recommended for Database Engine permissions (Microsoft, 2025a).

5.1 Authentication, users, roles and permission model

Results

Messages

	LoginName	LoginType	is_disabled	is_policy_checked	is_expiration_checked	default_database_name	PasswordLastSetTime
1	SB_AUDITOR01	SQL_LOGIN	0	1	0	SmartBankDB	2026-07-21 02:15:31.097
2	sb_customer01	SQL_LOGIN	0	1	0	SmartBankDB	2026-07-15 22:23:38.440
3	sb_customer02	SQL_LOGIN	0	1	0	SmartBankDB	2026-07-15 22:23:38.593
4	sb_customer03	SQL_LOGIN	0	1	0	SmartBankDB	2026-07-15 22:23:38.750
5	sb_dba01	SQL_LOGIN	0	1	0	SmartBankDB	2026-07-15 22:23:37.817
6	sb_manager01	SQL_LOGIN	0	1	0	SmartBankDB	2026-07-15 22:23:37.973
7	sb_officer01	SQL_LOGIN	0	1	0	SmartBankDB	2026-07-15 22:23:38.137
8	sb_officer02	SQL_LOGIN	0	1	0	SmartBankDB	2026-07-15 22:23:38.290

	RoleName	UserName	RoleCode	StaffID	CustomerID	IsActive
1	role_bank_manager	SB_MANAGER01	BANK_MANAGER	ST0002	NULL	1
2	role_bank_officer	SB_OFFICER01	BANK_OFFICER	ST0003	NULL	1
3	role_bank_officer	SB_OFFICER02	BANK_OFFICER	ST0004	NULL	1
4	role_customer	SB_CUSTOME...	CUSTOMER	NULL	CU0001	1
5	role_customer	SB_CUSTOME...	CUSTOMER	NULL	CU0002	1
6	role_customer	SB_CUSTOME...	CUSTOMER	NULL	CU0003	1
7	role_dba	SB_DBA01	DBA	ST0001	NULL	1
8	role_security_audi...	SB_AUDITOR01	SECURITY_AU...	NULL	NULL	1

	RoleName	PermissionState	PermissionName	PermissionClass	SecurableName
1	role_bank_manager	DENY	DELETE	SCHEMA	[bank]
2	role_bank_manager	DENY	INSERT	SCHEMA	[bank]
3	role_bank_manager	DENY	UPDATE	SCHEMA	[bank]
4	role_bank_manager	DENY	CREATE TAB...	DATABASE	SmartBankDB
5	role_bank_manager	GRANT	EXECUTE	OBJECT_OR...	[api].[usp_Get...
6	role_bank_manager	GRANT	EXECUTE	OBJECT_OR...	[api].[usp_Ma...
7	role_bank_manager	GRANT	EXECUTE	OBJECT_OR...	[api].[usp_Ma...
8	role_bank_manager	GRANT	EXECUTE	OBJECT_OR...	[api].[usp_Ma...

Figure 2. Configured SQL logins, role membership and role permissions.

Figure 2 shows the configured SQL logins SB_DBA01, SB_MANAGER01, SB_OFFICER01/02, SB_CUSTOMER01/02/03 and the additional SB_AUDITOR01 account. Each database user is mapped through sec.LoginIdentityMap to a RoleCode and, where applicable, a StaffID or CustomerID. Business roles receive explicit DENY permissions for direct INSERT/UPDATE/DELETE on the bank schema and are granted EXECUTE only on approved procedures and SELECT only on role-appropriate views. This design will help in centralizing the validation along with business rules and will not allow for granting broad inherited permissions.

Explicit GRANT, DENY and role membership is important because SQL Server checks permissions in a hierarchy of securables. The design intentionally grants a more restrictive view and procedure permissions and DENYs for unsafe operations on the base-table. This is because the data structures are separated from the controlled interfaces that the operational users interact with (Microsoft, 2026a).

5.2 Authorization Matrix, RBAC and Security Metadata

	MatrixID	RoleName	FunctionalArea	RequiredAccess	ImplementedUsing	DirectBaseTableDML
4	7	role_bank_officer	Own staff record	View and update own staff record except salary.	api.usp_GetMyStaffRecord and api.usp_Staff_U...	Denied
5	8	role_bank_officer	Customer accounts	View all accounts/transactions; onboard customer...	RLS-protected views, officer procedures, and U...	Denied
6	9	role_customer	Public staff directory	View all staff names and contact numbers.	SELECT on sec.vw_PublicStaffDirectory.	Denied
7	10	role_customer	Own profile	View own decrypted sensitive information.	api.usp_GetMyCustomerProfile automatically ope...	Denied
8	11	role_customer	Own accounts an...	View only own rows.	RLS policies on bank.Account and bank.Transac...	Denied
9	12	role_customer	Financial transacti...	Perform valid deposit, withdrawal and transfer only.	Transactional stored procedures with ownership. ...	Denied
10	1	role_dba	Public staff directory	View all staff names and contact numbers.	SELECT on sec.vw_PublicStaffDirectory.	Denied
11	2	role_dba	Own staff record	View and update own staff record except salary u...	api.usp_GetMyStaffRecord and api.usp_Staff_U...	Denied

	SecurityPolicy	is_enabled	predicate_type_desc	TargetSchema	TargetTable	predicate_definition
1	AccountAccessPolicy	1	BLOCK	bank	Account	((sec].[fn_AccountAccessPredicate])((CustomerID)))
2	AccountAccessPolicy	1	BLOCK	bank	Account	((sec].[fn_AccountAccessPredicate])((CustomerID)))
3	AccountAccessPolicy	1	FILTER	bank	Account	((sec].[fn_AccountAccessPredicate])((CustomerID)))
4	TransactionAccess...	1	BLOCK	bank	Transactio...	((sec].[fn_TransactionAccessPredicate])((Custom...
5	TransactionAccess...	1	BLOCK	bank	Transactio...	((sec].[fn_TransactionAccessPredicate])((Custom...
6	TransactionAccess...	1	FILTER	bank	Transactio...	((sec].[fn_TransactionAccessPredicate])((Custom...

	SchemaName	TableName	ColumnName	masking_function
1	bank	Customer	Email	email()
2	bank	Customer	Phone	partial(0, "XXX-XXX-", 4)
3	bank	Customer	DateOfBirth	default()
4	history	Customer...	Email	email()
5	history	Customer...	Phone	partial(0, "XXX-XXX-", 4)
6	history	Customer...	DateOfBirth	default()

	SchemaName	TableName	ColumnName	label	information_type	rank_desc
1	bank	Account	Balance	Private	Financial	HIGH
2	bank	Customer	AddressCipher	Private	Address	HIGH
3	bank	Customer	ICNumberCi...	Private	National ID	CRITICAL
4	bank	StaffPrivate	SalaryCipher	Private	Financial	HIGH
5	bank	Transacti...	Amount	Private	Financial	HIGH
6	history	Account...	Balance	Private	Financial	HIGH
7	history	Customer...	AddressCipher	Private	Address	HIGH
8	history	Customer...	ICNumberCi...	Private	National ID	CRITICAL

Figure 3. Security matrix, enabled RLS predicates, dynamic masks and sensitivity classifications.

Figure 3 combines the implemented authorisation matrix with RLS, DDM and sensitivity-classification metadata. The matrix documents each role's functional area, required access, implementation mechanism and whether direct base-table DML is allowed. For example, Bank Officers may service customers through approved views and procedures while direct bank-schema DML remains denied; Customers are restricted to their own profile, accounts and valid financial transactions; and the DBA receives table-definition rights without business-data access.

The enabled security policies apply FILTER and BLOCK predicates to bank.Account and bank.TransactionRecord. Managers and Officers are allowed operational visibility, while Customer sessions are matched to the owning CustomerID through sec.LoginIdentityMap. Because the rule is enforced by SQL Server rather than only by an application query, accidental direct SELECT permission would still be filtered by RLS (Microsoft, 2026b). The same evidence set also confirms active DDM and persistent SQL Server sensitivity labels (Microsoft, 2026d).

5.3 Row-Level Security

Two schema-bound predicate functions - `sec.fn_AccountAccessPredicate` and `sec.fn_TransactionAccessPredicate` - are attached to `sec.AccountAccessPolicy` and `sec.TransactionAccessPolicy`. FILTER predicates restrict reads and BLOCK predicates protect INSERT/UPDATE paths. `dbo` is allowed for administration; Bank Manager and Bank Officer identities may view operational rows; Customer identities are permitted only where the row `CustomerID` matches their mapped `CustomerID`. As shown in Figure 3, both policies are enabled and in Figure 7/8 it is shown that customers are isolated with different authenticated users.

5.4 Secure Views

Views are intentionally implemented security interfaces that are not necessarily convenience projections. They minimize the number of exposed columns, maintain the RLS/DDM behavior and make role permissions easier to reason about. [Selectively granted views by role:]

View	Primary user	Exposure	Security purpose
<code>sec.vw_PublicStaffDirectory</code>	All authenticated business users	StaffName and Phone only	Contains a design requirement that authenticated users can see staff name/contact but not other staff fields.
<code>sec.vw_StaffBasicAccess</code>	DBA/Manager/Officer	Role-filtered staff fields	Staff have their own record access, and the Manager also has access to the Bank Officers.
<code>sec.vw_AccountAccess</code>	Manager/Officer/Customer	Account data	RLS transparently decides what rows to display.
<code>sec.vw_TransactionAccess</code>	Manager/Officer/Customer	Transaction data	RLS will automatically identify which rows of transactions can be seen.
<code>sec.vw_CustomerContact</code>	Manager/Officer	Customer contact fields	DDM continues to work, Bank Officer is provided with UNMASK for servicing requirements.
<code>reporting.vw_BranchAccountSummary</code>	Manager	Aggregated branch/account statistics	Delivers management reporting but lacks comprehensive table manipulation.
<code>sec.vw_RoleUserMapping</code>	Security Auditor	Role/user mappings	Provides access to review and demonstration evidence.
<code>sec.vw_ExplicitRolePermissions</code>	Security Auditor	Explicit GRANT/DENY metadata	Supports RBAC Verification and Audit Review.

5.5 DBA Access Test

ResultsMessages

1	LoginName	DatabaseUser
1	SB_DBA01	SB_DBA01

	StaffName	Phone
1	Aman Rahman	03-21001001
2	Sti Nur Aisyah	03-21001002
3	Daniel Lim	03-21001999
4	Priya Nair	03-21001004

	StaffID	StaffName	Position	Branch	Phone	IsActive	ModifiedAt	ModifiedBy
1	ST0001	Aman Rahman	DBA	Kuala Lumpur HQ	03-21001001	1	2026-07-20 18:15:31.141	MicrosoftAccount\hemanth030304@gmail.com

	StaffID	StaffName	Position	Branch	Phone	Salary	LastSalaryReviewDate	IsActive	ModifiedAt
1	ST0001	Aman Rahman	DBA	Kuala Lumpur HQ	03-21001001	8500.00	2026-01-01	1	2026-07-20 18:15:31.141

	ExpectedErrorNumber	ExpectedErrorMessage
1	229	The SELECT permission was denied on the object 'v...

	ExpectedErrorNumber	ExpectedErrorMessage
1	229	The INSERT permission was denied on the object '...

Figure 4. DBA role functional and negative access tests.

The session is referred to as SB_DBA01 in figure 4. The DBA will be able to see the public staff directory and his/her own staff record; SQL Server error 229 will be returned for any unauthorised attempts to SELECT/INSERT business data. The DDL audit evidence shown in Figure 11 also captures actions taken by SB_DBA01 for CREATE_TABLE and DROP_TABLE on the bank business-data schema, which allows the DBA to manipulate the table definitions while being explicitly denied access to SELECT/INSERT/UPDATE/DELETE against the same schema.

5.6 Bank Manager Access and Staff-Management Test

	LoginName	DatabaseUser												
1	SB_MANAGER01	SB_MANAGER01												
	StaffName	Phone												
1	Amran Rahman	03-21001001												
2	Sti Nur Aisyah	03-21001002												
3	Daniel Lim	03-21001003												
4	Priya Nar	03-21001004												
	StaffID	StaffName	Position	Branch	Phone	IsActive	ModifiedAt	ModifiedBy						
1	ST0002	Sti Nur Aisyah	Bank Manager	Kuala Lumpur HQ	03-21001002	1	2026-07-20 18:15:31.141	MicrosoftAccount\hemarh030304@gmail.com						
2	ST0003	Daniel Lim	Bank Officer	Cheras Branch	03-21001003	1	2026-07-20 18:15:31.141	MicrosoftAccount\hemarh030304@gmail.com						
3	ST0004	Priya Nar	Bank Officer	Petaling Jaya Branch	03-21001004	1	2026-07-20 18:15:31.141	MicrosoftAccount\hemarh030304@gmail.com						
	StaffID	StaffName	Position	Branch	Phone	Salary	LastSalaryReviewDate	IsActive	ModifiedAt					
1	ST0002	Sti Nur Aisyah	Bank Manager	Kuala Lumpur HQ	03-21001002	12000.00	2026-01-01	1	2026-07-20 18:15:31.141					
	StaffID	StaffName	Position	Branch	Phone	Salary	LastSalaryReviewDate	IsActive	ModifiedAt					
1	ST0002	Sti Nur Aisyah	Bank Manager	Kuala Lumpur HQ	03-21001002	12000.00	2026-01-01	1	2026-07-20 18:15:31.141					
2	ST0003	Daniel Lim	Bank Officer	Cheras Branch	03-21001003	5200.00	2026-01-01	1	2026-07-20 18:15:31.141					
3	ST0004	Priya Nar	Bank Officer	Petaling Jaya Br.	03-21001004	5400.00	2026-01-01	1	2026-07-20 18:15:31.141					
	AccountID	CustomerID	AccountType	Balance	AccountStatus	OpenedAt	ClosedAt	ModifiedAt						
1	AC00000001	CU0001	Savings	5000.00	Active	2026-07-20 18:15:31.156	NULL	2026-07-20 18:15:31.156						
2	AC00000002	CU0001	Current	1500.00	Active	2026-07-20 18:15:31.156	NULL	2026-07-20 18:15:31.156						
3	AC00000003	CU0002	Savings	8000.00	Active	2026-07-20 18:15:31.156	NULL	2026-07-20 18:15:31.156						
4	AC00000004	CU0003	Savings	3200.00	Active	2026-07-20 18:15:31.156	NULL	2026-07-20 18:15:31.156						
	TransID	TransactionReference	TransferGroupID	CustomerID	AccountID	CounterpartyAccountID	TransDate	Amount	TransactionType	Direction	BalanceBefore	BalanceAfter	Remarks	InitiatedByLogin
1	1	345190FD-6684-F111-ADF6-580205C41827	NULL	CU0001	AC00000001	NULL	2026-07-10 18:15:31.167	5000.00	Deposit	Credit	0.00	5000.00	Opening deposit	SB_CUSTOMER01
2	2	385190FD-6684-F111-ADF6-580205C41827	NULL	CU0001	AC00000002	NULL	2026-07-11 18:15:31.167	1500.00	Deposit	Credit	0.00	1500.00	Opening deposit	SB_CUSTOMER01
3	3	3C5190FD-6684-F111-ADF6-580205C41827	NULL	CU0002	AC00000003	NULL	2026-07-12 18:15:31.167	8000.00	Deposit	Credit	0.00	8000.00	Opening deposit	SB_CUSTOMER02
4	4	3D5190FD-6684-F111-ADF6-580205C41827	NULL	CU0003	AC00000004	NULL	2026-07-13 18:15:31.167	3200.00	Deposit	Credit	0.00	3200.00	Opening deposit	SB_CUSTOMER03
	CustomerID	CustomerName	Email	Phone	DateOfBirth	CustomerStatus	ModifiedAt							
1	CU0001	Nur Izzati Hassan	jxxx@xxxx.com	xxx-xxx-6789	1900-01-01	Active	2026-07-20 18:15:31.143							
2	CU0002	Jason Wong	jxxx@xxxx.com	xxx-xxx-8990	1900-01-01	Active	2026-07-20 18:15:31.143							
3	CU0003	Kavitha Raman	kxxx@xxxx.c...	xxx-xxx-5667	1900-01-01	Active	2026-07-20 18:15:31.143							
	StaffID	StaffName	Position	Branch	Phone	Salary	LastSalaryReviewDate	IsActive	ModifiedAt					
1	ST0002	Sti Nur Aisyah	Bank Manager	Kuala Lumpur HQ	03-21001002	12000.00	2026-01-01	1	2026-07-20 18:15:31.141					
2	ST0003	Daniel Lim	Bank Officer	Cheras Branch	03-21001999	5300.00	2026-08-04	1	2026-08-04 18:23:30.536					
3	ST0004	Priya Nar	Bank Officer	Petaling Jaya Br.	03-21001004	5400.00	2026-01-01	1	2026-07-20 18:15:31.141					
	ExpectedErrorNumber	ExpectedErrorMessage												
1	229	The SELECT permission was denied on the object "..."												

Figure 5. Bank Manager role access test.

Figure 5 shows that SB_MANAGER01 is able to access the public staff directory, his/her own record, the records for all staff with the job role of Bank Officer, all customer accounts and all transactions. Where it isn't necessary to reveal full personal details to the customer, such information is hidden. The Manager-controlled update of Bank Officer ST0003 has been displayed with the Manager's salary changed to 5300.00 and an updated salary-review date, and direct unauthorised access to the tables returns error 229. This meets confidentiality (Manager visibility) and integrity (Staff management via the Manager role not table DML).

5.7 Bank Officer Access and Customer-Account Management Test

Results		Messages	
1	LoginName	DatabaseUser	
1	SB_OFFICER01	SB_OFFICER01	
1	StaffName	Phone	
2	Aiman Rahman	03-21001001	
3	Sti Nur Aisyah	03-21001002	
4	Daniel Lim	03-21001999	
5	Priya Nair	03-21001004	
1	StaffID	StaffName	Position
1	ST0003	Daniel Lim	Bank Officer
2	Branch	Phone	IsActive
2	Cheras Branch	03-21001999	1
3	ModifiedAt	ModifiedBy	
3	2026-08-04 18:23:30.536	SB_MANAGER01	
1	StaffID	StaffName	Position
1	ST0003	Daniel Lim	Bank Officer
2	Branch	Phone	Salary
2	Cheras Branch	03-21001999	5300.00
3	LastSalaryReviewDate	IsActive	ModifiedAt
3	2026-08-04	1	2026-08-04 18:23:30.536
1	AccountID	CustomerID	AccountType
1	AC00000001	CU0001	Savings
2	Balance	AccountStatus	OpenedAt
2	5000.00	Active	2026-07-20 18:15:31.156
3	ClosedAt	ModifiedAt	
3	NULL	2026-07-20 18:15:31.156	
4	AC00000002	CU0001	Current
4	1500.00	Active	2026-07-20 18:15:31.156
5	AC00000003	CU0002	Savings
5	8000.00	Active	2026-07-20 18:15:31.156
6	AC00000004	CU0003	Savings
6	3200.00	Active	2026-07-20 18:15:31.156
1	TransID	TransactionReference	TransferGroupID
1	1	3A519DFD-6684-F111-ADF6-580205C41827	NULL
2	CustomerID	AccountID	CounterpartyAccountID
2	CU0001	AC00000001	NULL
3	TransDate	Amount	TransactionType
3	2026-07-10 18:15:31.167	5000.00	Deposit
4	Direction	BalanceBefore	BalanceAfter
4	Credit	0.00	5000.00
5	Remarks	InitiatedByLogin	
5	Opening deposit	SB_CUSTOMER01	
6	2	3B519DFD-6684-F111-ADF6-580205C41827	NULL
6	CU0001	AC00000002	NULL
7	2026-07-11 18:15:31.167	1500.00	Deposit
8	Credit	0.00	1500.00
9	Opening deposit	SB_CUSTOMER01	
10	3	3C519DFD-6684-F111-ADF6-580205C41827	NULL
10	CU0002	AC00000003	NULL
11	2026-07-12 18:15:31.167	8000.00	Deposit
12	Credit	0.00	8000.00
13	Opening deposit	SB_CUSTOMER02	
14	4	3D519DFD-6684-F111-ADF6-580205C41827	NULL
14	CU0003	AC00000004	NULL
15	2026-07-13 18:15:31.167	3200.00	Deposit
16	Credit	0.00	3200.00
17	Opening deposit	SB_CUSTOMER03	
1	CustomerID	CustomerName	Email
1	CU0001	Nur Izzati Hassan	izzati@example.com
2	Phone	DateOfBirth	CustomerStatus
2	012-3456789	1998-04-12	Active
3	ModifiedAt		
3	2026-07-20 18:15:31.143		
4	CU0002	Jason Wong	jason.wong@example.com
4	013-7788990	1995-09-21	Active
5	CU0003	Kavitha Raman	kavitha.r@example.com
5	016-4455667	2000-02-18	Active
1	AccountID	CustomerID	AccountType
1	AC000000099	CU0099	Savings
2	Balance	AccountStatus	OpenedAt
2	0.00	Active	2026-08-05 14:33:15.097
3	ClosedAt	ModifiedAt	
3	NULL	2026-08-05 14:33:15.097	
1	ExpectedErrorNumber	ExpectedErrorMessage	
1	229	The SELECT permission was denied on the object '...	

Figure 6. Bank Officer role access test.

Figure 6 displays the public staff directory and the Officer's own staff record, all customer accounts, and all transactions and operational customer-contact data. In addition, the evidence reveals that an account (AC000000099) was opened for customer CU0099 via the Officer-controlled interface, thus meeting the integrity requirement of customer account management by Bank Officers. An unauthorised direct base-table query is denied, 229.

5.8 Customer 01 Access and Transaction Test

Results

Messages

1

SB_CUSTOMER01

SB_CUSTOMER01

StaffName

Phone

1

Aman Rahman

03-21001001

2

Siti Nur Aisyah

03-21001002

3

Daniel Lim

03-21001999

4

Priya Nar

03-21001004

CustomerID

CustomerName

Email

Phone

DateOfBirth

ICNumber

Address

CustomerStatus

ModifiedAt

1

CU0001

Nur Izzati Hassan

izzati@example.com

012-3456789

1998-04-12

980412-14-5678

12 Jalan Ampang, 50450 Kuala Lumpur

Active

2026-07-20 18:15:31.143

AccountID

CustomerID

Account Type

Balance

AccountStatus

OpenedAt

ClosedAt

ModifiedAt

1

AC00000001

CU0001

Savings

5000.00

Active

2026-07-20 18:15:31.156

NULL

2026-07-20 18:15:31.156

2

AC00000002

CU0001

Current

1500.00

Active

2026-07-20 18:15:31.156

NULL

2026-07-20 18:15:31.156

TransID

TransactionReference

TransferGroupID

CustomerID

AccountID

CounterpartyAccountID

TransDate

Amount

TransactionType

Direction

BalanceBefore

BalanceAfter

Remarks

InitiatedByLogin

1

3A5190FD-6684-F111-ADF6-580205C41827

NULL

CU0001

AC00000001

NULL

2026-07-10 18:15:31.167

5000.00

Deposit

Credit

0.00

5000.00

Opening deposit

SB_CUSTOMER01

2

3B5190FD-6684-F111-ADF6-580205C41827

NULL

CU0001

AC00000002

NULL

2026-07-11 18:15:31.167

1500.00

Deposit

Credit

0.00

1500.00

Opening deposit

SB_CUSTOMER01

AccountID

CustomerID

Account Type

Balance

AccountStatus

OpenedAt

ClosedAt

ModifiedAt

1

AC00000001

CU0001

Savings

5035.00

Active

2026-07-20 18:15:31.156

NULL

2026-09-05 14:45:24.938

2

AC00000002

CU0001

Current

1500.00

Active

2026-07-20 18:15:31.156

NULL

2026-07-20 18:15:31.156

TransID

TransactionReference

TransferGroupID

CustomerID

AccountID

CounterpartyAccountID

TransDate

Amount

TransactionType

Direction

BalanceBefore

BalanceAfter

Remarks

IdempotencyKey

InitiatedByLogin

1

5 E7CA2F4A-DC90-F111-ADF6-580205C41827

NULL

CU0001

AC00000001

NULL

2026-08-05 14:45:24.704

50.00

Deposit

Credit

5000.00

5050.00

Demo deposit

650BC64B-2F14-4376-9B6C-6C3CD73595A3

SB_CUSTOMER01

TransID

TransactionReference

TransferGroupID

CustomerID

AccountID

CounterpartyAccountID

TransDate

Amount

TransactionType

Direction

BalanceBefore

BalanceAfter

Remarks

IdempotencyKey

InitiatedByLogin

1

6 E8CA2F4A-DC90-F111-ADF6-580205C41827

NULL

CU0001

AC00000001

NULL

2026-08-05 14:45:24.818

10.00

Withdrawal

Debit

5050.00

5040.00

Demo withdrawal

91142A3A-39FA-46B5-842B-46C4558D55D9

SB_CUSTOMER01

TransID

TransactionReference

TransferGroupID

CustomerID

AccountID

CounterpartyAccountID

TransDate

Amount

TransactionType

Direction

BalanceBefore

BalanceAfter

Remarks

IdempotencyKey

InitiatedByLogin

1

7 E9CA2F4A-DC90-F111-ADF6-580205C41827

2A8F4E55-84C6-4CAC-A4DD-61AB711D4168

CU0001

AC00000001

AC00000003

2026-08-05 14:45:24.940

5.00

Transfer

Debit

5040.00

5035.00

Demo transfer

9E57640B-0CE8-4A8B-8005-000000000000

SB_CUSTOMER01

2

8 EACA2F4A-DC90-F111-ADF6-580205C41827

2A8F4E55-84C6-4CAC-A4DD-61AB711D4168

CU0002

AC00000003

AC00000001

2026-08-05 14:45:24.941

5.00

Transfer

Credit

8000.00

8005.00

Demo transfer

NULL

SB_CUSTOMER01

AccountID

CustomerID

Account Type

Balance

AccountStatus

OpenedAt

ClosedAt

ModifiedAt

1

AC00000001

CU0001

Savings

5035.00

Active

2026-07-20 18:15:31.156

NULL

2026-09-05 14:45:24.938

2

AC00000002

CU0001

Current

1500.00

Active

2026-07-20 18:15:31.156

NULL

2026-07-20 18:15:31.156

TransID

TransactionReference

TransferGroupID

CustomerID

AccountID

CounterpartyAccountID

TransDate

Amount

TransactionType

Direction

BalanceBefore

BalanceAfter

Remarks

InitiatedByLogin

1

7 E9CA2F4A-DC90-F111-ADF6-580205C41827

2A8F4E55-84C6-4CAC-A4DD-61AB711D4168

CU0001

AC00000001

AC00000003

2026-08-05 14:45:24.940

5.00

Transfer

Debit

5040.00

5035.00

Demo transfer

SB_CUSTOMER01

2

6 E8CA2F4A-DC90-F111-ADF6-580205C41827

NULL

CU0001

AC00000001

NULL

2026-08-05 14:45:24.818

10.00

Withdrawal

Debit

5050.00

5040.00

Demo withd...

SB_CUSTOMER01

ExpectedErrorNumber

ExpectedErrorMessage

1

51307

Insufficient funds for the withdrawal.

ExpectedErrorNumber

ExpectedErrorMessage

1

229

The SELECT permission was denied on the object '...

Figure 7. Customer 01 row-level access and transaction operations.

In Figure 7, only the transactions related to CU0001, only the rows in the public staff directory and only the decrypted CU0001 profile are passed to SB_CUSTOMER01. Deposit, withdrawal and transfer operations are processed through the customer procedures and a deliberately excessive withdrawal results in the customer receiving a custom error 51307 (insufficient funds) and an unauthorised direct SELECT results in a customer error 229. This one test thus provides authentication, RLS, authorised decryption, as well as valid-transaction processing and invalid-transaction rejection and least-privilege enforcement.

5.9 Customer 02 Isolation Test

Results		Messages	
1	SB_CUSTOMER02	SB_CUSTOMER02	
CustomerID		CustomerName	Email
1	CU0002	Jason Wong	jason.wong@example.com
Phone		DateOfBirth	ICNumber
013-7788990		1995-09-21	950921-10-2244
Address		CustomerStatus	ModifiedAt
88 Jalan Kuchai Lama, 58200 Kuala Lumpur		Active	2025-07-20 18:15:31.143
AccountID		CustomerID	AccountType
1	AC00000003	CU0002	Savings
Balance		AccountStatus	OpenedAt
8005.00		Active	2025-07-20 18:15:31.156
ClosedAt		ModifiedAt	
NULL		2025-08-05 14:45:24.939	
TransID		TransactionReference	TransferGroupID
1	3	3C5190FD-6684-F111-ADF6-580205C41827	NULL
CustomerID		AccountID	CounterpartyAccountID
2	8	CU0002	AC00000003
TransDate		Amount	TransactionType
2025-07-12 18:15:31.167		8000.00	Deposit
Direction		BalanceBefore	BalanceAfter
Credit		0.00	8000.00
Remarks		InitiatedByLogin	
Opening deposit		SB_CUSTOMER02	
TransDate		Amount	TransactionType
2025-08-05 14:45:24.941		5.00	Transfer
Direction		BalanceBefore	BalanceAfter
Credit		8000.00	8005.00
Remarks		InitiatedByLogin	
Demo transfer		SB_CUSTOMER01	

Figure 8. Customer 02 sees only CU0002 data.

Figure 8 illustrates the results of a separate SB_CUSTOMER02 session requesting the CU0002 profile, account AC00000003 and only the rows for transactions relevant to CU0002. The transfer from CU0001 will only be seen as credit corresponding to it for CU0002. A comparison of Figure 7 and 8 is practical proof of the different set of rows returned when the same secured views are queried based on different authenticated customer identities.

5.10 Encryption Design and Ciphertext Storage

Results

Messages

StaffID	SalaryCipher	CipherBytes	
1	ST0001	0x00B94278C6C9804ABCA53E4452DECF1020000002AB2D87...	84
2	ST0002	0x00B94278C6C9804ABCA53E4452DECF10200000077FBEB...	84
3	ST0003	0x00B94278C6C9804ABCA53E4452DECF10200000048DBAE...	84
4	ST0004	0x00B94278C6C9804ABCA53E4452DECF10200000082BDA10...	84

CustomerID	ICNumberCipher	AddressCipher	ICNumberHash	HashSalt
1	CU0001	0x00B94278C6C9804ABCA53E4452DECF10200000042B0E35...	0x00B94278C6C9804ABCA53E4452DECF102000000732A9C6...	0xC324F0EDFA0BAEF5F762AF88C9792E55CCED664FF8C87B...
2	CU0002	0x00B94278C6C9804ABCA53E4452DECF1020000002151D6A...	0x00B94278C6C9804ABCA53E4452DECF102000000726CE0...	0x71BF61191806AF8527C9F240C1AD8C0ABC469A2671F5985...
3	CU0003	0x00B94278C6C9804ABCA53E4452DECF10200000090A1A5...	0x00B94278C6C9804ABCA53E4452DECF1020000004187E0E...	0x27709C2FD2E2BAAD85FB0C062A8760BF81BF40919DEAEA...
4	CU0009	0x00B94278C6C9804ABCA53E4452DECF10200000006E22EB...	0x00B94278C6C9804ABCA53E4452DECF10200000095C32DE...	0x53B740EACB668C960CA28D73A3395395CC88B187C538969...

CustomerID	SHA256Bytes	ICNumberHashHex
1	CU0001	32 C324F0EDFA0BAEF5F762AF88C9792E55CCED664FF8C87B...
2	CU0002	32 71BF61191806AF8527C9F240C1AD8C0ABC469A2671F5985B7...
3	CU0003	32 27709C2FD2E2BAAD85FB0C062A8760BF81BF40919DEAEA75...
4	CU0009	32 53B740EACB668C960CA28D73A3395395CC88B187C5389697F...

CustomerToken	CustomerAlias	EmailDomain	PhoneMasked	AgeBand	AccountCount	TotalBalanceBand	SnapshotUtc	
1	D43D31BA3E0722DE62E5CC6CCA2DA0A0ED0CC6C64C25002A...	Customer-0001	example.com	XXXXXXX5789	21-30	2	5,000-19,999	2026-08-05 17:07:02.744
2	833FB542E44B1CFBF7D68FDBD4A0CCE3D0FFFA387E830D36E...	Customer-0002	example.com	XXXXXXX3990	31-45	1	5,000-19,999	2026-08-05 17:07:02.744
3	77FB8E951B1F8F36217A1C971161879654C83C7EDD81F361BC50...	Customer-0003	example.com	XXXXXXX5667	21-30	1	Below 5,000	2026-08-05 17:07:02.744
4	0FC79F9D6708BF7F8539FD08E95E7441828D6FE263A8671AC0...	Customer-0009	example.com	XXXXXXX0099	21-30	1	Below 5,000	2026-08-05 17:07:02.744

CustomerToken	CustomerAlias	EmailDomain	PhoneMasked	AgeBand	AccountCount	TotalBalanceBand	SnapshotUtc	
1	D43D31BA3E0722DE62E5CC6CCA2DA0A0ED0CC6C64C25002A...	Customer-0001	example.com	XXXXXXX5789	21-30	2	5,000-19,999	2026-08-05 17:07:02.744
2	833FB542E44B1CFBF7D68FDBD4A0CCE3D0FFFA387E830D36E...	Customer-0002	example.com	XXXXXXX3990	31-45	1	5,000-19,999	2026-08-05 17:07:02.744
3	77FB8E951B1F8F36217A1C971161879654C83C7EDD81F361BC50...	Customer-0003	example.com	XXXXXXX5667	21-30	1	Below 5,000	2026-08-05 17:07:02.744
4	0FC79F9D6708BF7F8539FD08E95E7441828D6FE263A8671AC0...	Customer-0009	example.com	XXXXXXX0099	21-30	1	Below 5,000	2026-08-05 17:07:02.744

SchemaName	TableName	ColumnName	masking_function
1	bank	Customer	Email email()
2	bank	Customer	Phone partial(0, "XXX-XXX-", 4)
3	bank	Customer	DateOfBirth default()
4	history	Customer...	Email email()
5	history	Customer...	Phone partial(0, "XXX-XXX-", 4)
6	history	Customer...	DateOfBirth default()

Figure 9. Encrypted columns, salted hashes, masked analytics and mask metadata.

As can be seen from figure 9, staff salary, customer IC number and customer address are stored as ciphertext instead of plaintext. There's an encryption hierarchy: Database Master Key -> SmartBankColumnCertificate -> SmartBankColumnKey (AES-256). The following fields are encrypted by EncryptByKey, using the corresponding StaffID or CustomerID as an authenticator: SalaryCipher, ICNumberCipher and AddressCipher. This is in addition to the certificate which is used to encrypt the backup files and provides added protection for the sensitive columns and backup media.

5.11 Dynamic Masking, Static Masking, Hashing and Anonymization

Figure 9 also shows three types of privacy technique. Dynamic Data Masking doesn't alter the value stored in the table, just what is presented to the less privileged user; static masking/anonymisation generates a separate reduced-exposure analytical snapshot of the data; and hashing generates a non-reversible comparison value, not reversible confidentiality. These controls are not meant to be used as a replacement for encryption; they are designed to be used in conjunction with each other.

Technique	Implementation	Purpose
Dynamic Data Masking	bank.Customer.Email = email(); Phone = partial(...); DateOfBirth = default()	Reduces exposure in operational query results. Manager remains masked; Bank Officer receives UNMASK only for servicing needs.
Static masking / anonymisation	reporting.CustomerAnalyticsMasked	Stores CustomerToken, generic CustomerAlias, EmailDomain, PhoneMasked, AgeBand and TotalBalanceBand instead of full personal detail.
Salted hashing	Customer.ICNumberHash with per-customer HashSalt	SHA2-256 provides one-way duplicate/integrity support without storing a searchable plaintext IC number.
Pseudonymous analytics token	SHA2-256 of HashSalt + CustomerID	Supports repeatable analytical grouping without using the direct customer identifier in the reporting snapshot.

5.12 Automatic Authorised Decryption

Results Messages

	CustomerID	CustomerName	Email	Phone	DateOfBirth	ICNumber	Address	CustomerStatus	ModifiedAt
1	CU0001	Nur Izzati Hassan	izzati@example.com	012-3456789	1998-04-12	980412-14-5678	12 Jalan Ampang, 50450 Kuala Lumpur	Active	2026-07-20 18:15:31.143

Figure 10. Authorized customer profile returned in decrypted form.

Figure 10 shows the correct customer receiving CU0001 with readable IC number and address values. The decryption occurs inside `api.usp_GetMyCustomerProfile` after identifying the caller through `ORIGINAL_LOGIN()/sec.LoginIdentityMap` and opening `SmartBankColumnKey`. The Customer is not granted direct `SELECT` permission on the ciphertext columns. This directly satisfies the live-demo requirement that encrypted table data is automatically decrypted only for the appropriate authorised user.

DDM is a presentation-layer protection, not encryption: it changes query output while leaving the stored value unchanged. SmartBank therefore uses DDM for lower-risk contact-data exposure and uses AES-256 ciphertext for the most sensitive values such as salary, national ID and address (Microsoft, 2026c).

5.13 Additional Security Auditor Role

The assignment has four operational roles, but a separation-of-duties enhancement that has been added to the implementation includes `role_security_auditor` and `SB_AUDITOR01`. The auditor has no bank-schema DML and no direct access to the business-table, but has read-only access to the audit schema, the data dictionary, the authorisation matrix, the role/user mapping, explicit role permissions, masked analytical data and the SQL Server Audit reader procedure. This helps to review security evidence independently without being able to modify banking data.

6. Integrity Implementation and Evidence

It enforces integrity by prohibiting any base-table manipulation, routing business changes via role-specific processes, and auditing financial rules and recording actions at several layers of integrity. Modification responsibilities of the assignment are carried out directly: DBAs don't modify business data, Managers don't modify staff, Officers don't modify customer/account records, each staff member can modify allowed parts of his/her own staff record (not salary) and Customers only execute deposit, withdrawal and transfer procedures.

6.1 Controlled DML and separation of duties

Figures 2 and 3 illustrate the separation of duties model, which prohibits operational roles at the bank schema from performing direct INSERT/UPDATE/DELETE operations and only allows EXECUTE for specific stored procedures, which have been approved. This means that validation, ownership checks, transaction handling, encryption/decryption, audit and error handling still happen all in the same place, the controlled interfaces, which prevents them from being circumvented by ad hoc table DML.

6.2 Role-Specific Modification Responsibilities

The following controls specify how the assignment integrity rules are to be implemented as procedures and permission. The self-service procedure does not contain a salary parameter; this is deliberate as staff cannot update salary via their self-service, and direct StaffPrivate updates continue to be blocked.

Role	Allowed modification	Implemented control	Evidence
DBA	Create/manage table definitions	CREATE TABLE + ALTER on bank/history; business SELECT/INSERT/UPDATE/DELETE denied	Figure 11 records DBA CREATE_TABLE/DROP_TABLE; Figure 4 shows business-data denial.
Bank Manager	Create/update Bank Officers and salary	api.usp_Manager_CreateBankOfficer; api.usp_Manager_UpdateBankOfficer	Figure 5 shows Officer ST0003 salary/details update.
Bank Officer	Create/update customers and accounts	api.usp_Officer_CreateCustomer; api.usp_Officer_UpdateCustomer; api.usp_Officer_CreateAccount; api.usp_Officer_UpdateAccountStatus	Figure 6 shows AC00000099 created for CU0099.
Staff (self)	Update own non-salary details	api.usp_Staff_UpdateOwnRecord; no salary parameter; direct private-table update denied	Implementation present; final self-update screenshot recommended before submission.
Customer	Deposit, withdraw and transfer	api.usp_Customer_Deposit / Withdraw / Transfer	Figure 7 proves all three valid transaction types and an invalid withdrawal.

6.3 Valid and Invalid Transaction Controls

Figure 7 shows all three valid transaction types required by the assignment: deposit, withdrawal and transfer. Each procedure validates caller role, ownership, account status, positive amount and resulting balance, uses an explicit SQL transaction with XACT_ABORT, and writes transaction rows containing BalanceBefore and BalanceAfter. A deliberately excessive withdrawal is rejected with error 51307 and the failed attempt is written to audit.BusinessActionLog. The design therefore combines preventive validation with detective audit evidence.

6.4 Idempotency and Financial Transaction Traceability

Financial procedures take an IdempotencyKey and bank. There is a unique filtered index on that key, called TransactionRecord. If a request is attempted again, the same transaction is returned, rather than a new duplicate. Additional traceability for reconciliation and investigation provided by TransactionReference, TransferGroupID, CustomerID, AccountID, CounterpartyAccountID, BalanceBefore, BalanceAfter, InitiatedByLogin and SourceApplication. This is an extra banking-specific integrity constraint, on top of the minimum assignment schema.

6.5 Audit Strategy and Audit Matrix

The audit design deliberately uses overlapping native and custom mechanisms so that both technical events and business context are preserved. audit.AuditMatrix stores the audit area, event scope, mechanism, storage location, reviewer and retention guideline. This gives the Security Auditor a documented basis for demonstrating why each event is collected and where its evidence is retained.

Audit area	Events	Mechanism	Evidence store	Guideline
Login/logout	Successful login, failed login, logout	SQL Server Audit server action groups	.sqlaudit files	>=90 days for assignment evidence
Data access	SELECT and procedure execution	Database Audit Specification	.sqlaudit files	>=90 days
Valid DML	INSERT/UPDATE/DELETE + before/after images	AFTER DML triggers + SQL Server Audit	audit.DMLChangeLog + .sqlaudit	>=1 year
Invalid business action	Invalid amount, balance, ownership, role, etc.	Procedure TRY/CATCH logging	audit.BusinessActionLog	>=1 year
Invalid direct DML	Permission-denied or RLS-blocked writes	SQL Server Audit (succeeded = 0)	.sqlaudit files	>=1 year
DDL	CREATE/ALTER/DROP and definition changes	Database DDL trigger + SQL Server Audit	audit.DDLSecurityChangeLog + .sqlaudit	>=1 year
Permission changes	GRANT/DENY/REVOKE; users/roles/membership	DDL trigger + native audit	audit.DDLSecurityChangeLog + .sqlaudit	>=1 year

Backup operations	FULL/DIFF/LOG result, encryption, verification	Backup procedure + msdb job/backup history	audit.BackupEvidence + msdb	Retain enough generations for RPO/restore tests
-------------------	--	--	-----------------------------	---

6.6 Data Manipulation Language (DML) Audit

Successful INSERT, UPDATE and DELETE operations on bank.Staff, bank.StaffPrivate, bank.Customer, bank.Account and bank.TransactionRecord are captured by AFTER triggers running WITH EXECUTE AS OWNER. Each record in audit.DMLChangeLog stores the original login, mapped database user, schema/table, action, affected-row count, host/application and before/after row images as JSON. This provides detailed change evidence independently of the higher-level business-action audit.

6.7 Business-Action, DDL and SQL Server Audit Evidence

Results	Messages	LogName	Database/UserName	RoleCode	ActionCategory	ActionName	TargetObject	Success	ErrorNumber	ErrorMessage
1	20	2026-08-05 17:18:46.562	SB_CUSTOMER01	CUSTOMER	DATA_ACCESS	CUSTOMER_VIEW_OWN_DECRYPTED_PROFILE	bank.Customer	1	NULL	NULL
2	19	2026-08-05 17:07:02.754	MicrosoftAccount\hemanth030304@gmail.com	dbo	NULL	REFRESH_STATIC_MASKED_ANALYTICS	reporting.CustomerAnalyticsMasked	1	NULL	NULL
3	18	2026-08-05 14:48:42.028	SB_CUSTOMER02	CUSTOMER	DATA_ACCESS	CUSTOMER_VIEW_OWN_DECRYPTED_PROFILE	bank.Customer	1	NULL	NULL
4	17	2026-08-05 14:45:24.949	SB_CUSTOMER01	CUSTOMER	TRANSACTION	CUSTOMER_WITHDRAWAL	bank.Account/bank.TransactionRecord	0	51307	Insufficient funds
5	16	2026-08-05 14:45:24.942	SB_CUSTOMER01	CUSTOMER	TRANSACTION	CUSTOMER_TRANSFER	bank.Account/bank.TransactionRecord	1	NULL	NULL
6	15	2026-08-05 14:45:24.920	SB_CUSTOMER01	CUSTOMER	TRANSACTION	CUSTOMER_WITHDRAWAL	bank.Account/bank.TransactionRecord	1	NULL	NULL
7	14	2026-08-05 14:45:24.761	SB_CUSTOMER01	CUSTOMER	TRANSACTION	CUSTOMER_DEPOSIT	bank.Account/bank.TransactionRecord	1	NULL	NULL
8	13	2026-08-05 14:45:24.544	SB_CUSTOMER01	CUSTOMER	DATA_ACCESS	CUSTOMER_VIEW_OWN_DECRYPTED_PROFILE	bank.Customer	1	NULL	NULL

ChangeAuditID	EventTimeUtc	LogName	Database/UserName	SchemaName	TableName	DMLAction	RowCountAffected	OldRowJson	NewRowJson	HostName	ApplicationName
1	68	2026-08-05 14:31:58.436	SB_DBA01	SB_DBA01	DROP_TABLE	TABLE	bank	DBA_DemoTable	DROP TABLE bank.DBA_DemoTable	HEMANTHPC	Microsoft SQL Server Management Studio - Query
2	67	2026-08-05 14:31:58.426	SB_DBA01	SB_DBA01	CREATE_TABLE	TABLE	bank	DBA_DemoTable	CREATE TABLE bank.DBA_DemoTable (DemoID int ...	HEMANTHPC	Microsoft SQL Server Management Studio - Query
3	66	2026-08-04 18:09:14.049	SB_DBA01	SB_DBA01	DROP_TABLE	TABLE	bank	DBA_DemoTable	DROP TABLE bank.DBA_DemoTable	HEMANTHPC	Microsoft SQL Server Management Studio - Query
4	65	2026-08-04 18:09:14.039	SB_DBA01	SB_DBA01	CREATE_TABLE	TABLE	bank	DBA_DemoTable	CREATE TABLE bank.DBA_DemoTable (DemoID int ...	HEMANTHPC	Microsoft SQL Server Management Studio - Query
5	64	2026-07-23 14:06:35.816	Microsoft...	dbo	CREATE_PROC	PROCED	admin	usp_BackupRe...	CREATE OR ALTER PROCEDURE admin.usp_Backup...	HEMANTHPC	Microsoft SQL Server Management Studio - Query
6	63	2026-07-20 18:15:31.829	Microsoft...	dbo	CREATE_PROC	PROCED	admin	usp_RunBackup	CREATE OR ALTER PROCEDURE admin.usp_RunBackup...	HEMANTHPC	Microsoft SQL Server Management Studio - Query
7	62	2026-07-20 18:15:31.705	Microsoft...	dbo	GRANT_DATA	DATABASE	SmartBankDB	SmartBankDB	GRANT VIEW DEFINITION ON DATABASE: SmartBank...	HEMANTHPC	Microsoft SQL Server Management Studio - Query
8	61	2026-07-20 18:15:31.703	Microsoft...	dbo	GRANT_DATA	PROCED	audit	usp_ReadSaSe...	GRANT EXECUTE ON OBJECT: audit.usp_ReadSaSe...	HEMANTHPC	Microsoft SQL Server Management Studio - Query

event_time	sequence_number	action_id	succeeded	permission_bitmask	is_column_permission	session_id	server_principal_name	session_server_principal_name	database_principal_name	target_server_principal_name	target_db
1	2026-08-05 17:20:14.4472830	1	LOGO	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
2	2026-08-05 17:20:14.4472830	1	LGIS	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
3	2026-08-05 17:20:14.3812487	1	LOGO	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
4	2026-08-05 17:20:14.3812487	1	LGIS	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
5	2026-08-05 17:20:14.3049660	1	LOGO	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
6	2026-08-05 17:20:14.3049660	1	LGIS	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
7	2026-08-05 17:20:14.1202102	1	LOGO	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
8	2026-08-05 17:20:14.1202102	1	LGIS	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
9	2026-08-05 17:20:14.0835620	1	LOGO	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
10	2026-08-05 17:20:14.0825630	1	LOGO	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
11	2026-08-05 17:20:14.0212799	1	LGIS	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
12	2026-08-05 17:20:12.5455855	1	LOGO	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
13	2026-08-05 17:20:12.5451527	1	LGIS	1	0x00000000000000000000000000000000	0	53	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
14	2026-08-05 17:20:12.8629400	1	LGIS	1	0x00000000000000000000000000000000	0	52	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
15	2026-08-05 17:19:21.3075098	1	LOGO	1	0x00000000000000000000000000000000	0	73	HemantHPC\hemant	MicrosoftAccount\hemanth030304@gmail.com		
16	2026-08-05 17:18:46.5552772	1	SL	1	0x00000000000000000000000000000001	1	72	sa	SB_CUSTOMER01	dbo	
17	2026-08-05 17:18:46.5457652	1	AS	1	0x00000000000000000000000000000000	0	72	sa	SB_CUSTOMER01	dbo	
18	2026-08-05 17:18:46.5457623	1	OP	1	0x00000000000000000000000000000000	0	72	sa	SB_CUSTOMER01	dbo	

Figure 11. Three-layer audit evidence for business actions, DDL and server events.

Figure 11 demonstrates the layered audit model. audit.BusinessActionLog records business/security operations with success/failure and error information, including CUSTOMER_VIEW_OWN_DECRYPTED_PROFILE, CUSTOMER_DEPOSIT, CUSTOMER_WITHDRAWAL and CUSTOMER_TRANSFER. audit.DDLSecurityChangeLog records definition changes such as CREATE_TABLE and DROP_TABLE together with command text, actor, host and application. The lower result set shows native SQL Server Audit records. Combining native auditing with application-specific audit records provides complementary technical and business-level accountability (Microsoft, 2026e).

SQL Server Audit records server- and database-level events to protected audit files and is used for successful/failed logins, logout, DML/SELECT/procedure execution, DDL and permission changes. The custom tables add context that native audit alone does not provide, such as business action names,

validation errors and before/after row images. Together these layers support investigation of both valid and invalid manipulations.

6.8 Permission and Role-Change Audit

Results

Messages

RoleName	UserName	RoleCode	StaffID	CustomerID	IsActive
role_bank_officer	SB_OFFICER03	NULL	NULL	NULL	NULL

DDLAuditID	Event TimeUTC	LoginName	DatabaseUserName	EventType	ObjectType	SchemaName	ObjectName	TSQLCommand	HostName	ApplicationName
72	2026-08-05 18:33:58.127	Microsoft Account\hemanth030304@gmail.com	dbo	REVOKE_DATABASE	TABLE	reporting	CustomerAnalyticsMasked	REVOKE SELECT ON OBJECT:reporting.CustomerAnalyticsM...	HEMANTHPC	Microsoft SQL Server N...
71	2026-08-05 18:33:58.125	Microsoft Account\hemanth030304@gmail.com	dbo	GRANT_DATABASE	TABLE	reporting	CustomerAnalyticsMasked	GRANT SELECT ON OBJECT:reporting.CustomerAnalyticsM...	HEMANTHPC	Microsoft SQL Server N...
70	2026-08-05 18:33:58.101	Microsoft Account\hemanth030304@gmail.com	dbo	ADD_ROLE_MEMBER	SQL USER	NULL	SB_OFFICER03	ALTER ROLE role_bank_officer ADD MEMBER SB_OFFICE...	HEMANTHPC	Microsoft SQL Server N...
69	2026-08-05 18:33:58.095	Microsoft Account\hemanth030304@gmail.com	dbo	CREATE_USER	SQL USER	NULL	SB_OFFICER03	CREATE USER SB_OFFICER03 FOR LOGIN SB_OFFICER03;	HEMANTHPC	Microsoft SQL Server N...
60	2026-07-20 18:15:31.702	Microsoft Account\hemanth030304@gmail.com	dbo	GRANT_DATABASE	TABLE	reporting	CustomerAnalyticsMasked	GRANT SELECT ON OBJECT:reporting.CustomerAnalyticsM...	HEMANTHPC	Microsoft SQL Server N...

incc_number	action_id	succeeded	permission_bitmask	is_column_permission	session_id	server_principal_name	session_server_principal_name	database_principal_name	target_server_principal_name	target_database_principal_name	database_...
1	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
2	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
3	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
4	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
5	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
6	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
7	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
8	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
9	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
10	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
11	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
12	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
13	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
14	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
15	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
16	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
17	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
18	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				
19	LGO	1	0x00000000000000000000000000000000	0	81	HenarthPC\heman	Microsoft Account\hemanth030304@gmail.com				

Query executed successfully.

HEMANTHPC (17.0 RTM) HEMANTHPC(heman (79)) SmartBankDB 00:00:0 Row: 2 Col: 15 300 row

Figure 12. Creation of an additional Bank Officer and associated audit trail.

Figure 12 shows a newly mapped SB_OFFICER03 account in role_bank_officer and audit records for security changes associated with the new user. The DDL/security audit contains CREATE_USER, ADD_ROLE_MEMBER, GRANT_DATABASE and REVOKE_DATABASE events. This directly prepares the system for the lecturer's live requirement to add users or alter permissions and then prove that the security change was captured.

6.9 Login, Logout and Failed-Login Audit

event_time	action_id	succeeded	server_principal_name	session_server_principal_name	database_name	statement	application_name	client_ip
2026-08-05 18:29:25.6428314	LGO	1	HemanthPC\heman	MicrosoftAccount\hemanth030304@gmail.com		-- network protocol: LPC set quoted_identifier on set...	Microsoft SQL Server Management Studio - Transa...	local machine
2026-08-05 18:29:25.6428314	LGIS	1	HemanthPC\heman	MicrosoftAccount\hemanth030304@gmail.com		-- network protocol: LPC set quoted_identifier on set...	Microsoft SQL Server Management Studio - Transa...	local machine
2026-08-05 18:29:25.5831798	LGIS	1	HemanthPC\heman	MicrosoftAccount\hemanth030304@gmail.com		-- network protocol: LPC set quoted_identifier on set...	Microsoft SQL Server Management Studio - Transa...	local machine
2026-08-05 18:29:24.5382070	LGO	1	HemanthPC\heman	MicrosoftAccount\hemanth030304@gmail.com		-- network protocol: LPC set quoted_identifier on set...	Microsoft SQL Server Management Studio - Query	local machine
2026-08-05 18:29:24.4819491	LGIS	1	HemanthPC\heman	MicrosoftAccount\hemanth030304@gmail.com		-- network protocol: LPC set quoted_identifier on set...	Microsoft SQL Server Management Studio - Query	local machine
2026-08-05 18:29:16.1003310	LGO	1	sb_customer01	SB_CUSTOMERO1			SQL Server Management Studio	local machine
2026-08-05 18:29:05.9176960	LGIS	1	HemanthPC\heman	MicrosoftAccount\hemanth030304@gmail.com		-- network protocol: LPC set quoted_identifier on set...	SQL Server Management Studio	local machine
2026-08-05 18:28:20.4692853	LGIF	0	SB_CUSTOMERO1			Login failed for user 'SB_CUSTOMERO1': Reason: P...	SQL Server Management Studio	local machine
2026-08-05 18:28:16.9542054	LGIF	0	SB_CUSTOMERO1			Login failed for user 'SB_CUSTOMERO1': Reason: P...	SQL Server Management Studio	local machine
2026-08-05 18:28:14.8133349	LGIF	0	SB_CUSTOMERO1			Login failed for user 'SB_CUSTOMERO1': Reason: P...	SQL Server Management Studio	local machine
2026-08-05 18:28:08.1117663	LGO	1	NT SERVICE\SQLTELEMETRY	NT Service\SQLTELEMETRY		-- network protocol: LPC set quoted_identifier on set...	SQLServerCEIP	local machine
2026-08-05 18:28:08.1117663	LGIS	1	NT SERVICE\SQLTELEMETRY	NT Service\SQLTELEMETRY		-- network protocol: LPC set quoted_identifier on set...	SQLServerCEIP	local machine
2026-08-05 18:28:08.0635407	LGIS	1	NT SERVICE\SQLTELEMETRY	NT Service\SQLTELEMETRY		-- network protocol: LPC set quoted_identifier on set...	SQLServerCEIP	local machine
2026-08-05 18:28:05.0606355	LGO	1	NT SERVICE\SQLTELEMETRY	NT Service\SQLTELEMETRY		-- network protocol: LPC set quoted_identifier on set...	SQLServerCEIP	local machine
2026-08-05 18:28:03.8438500	LGO	1	sb_customer01	SB_CUSTOMERO1			SQL Server Management Studio	local machine
2026-08-05 18:26:31.5221703	LGO	1	NT SERVICE\SQLSERVERAGENT	NT Service\SQLSERVERAGENT			SQLAgent - Schedule Saver	local machine
2026-08-05 18:26:31.5089558	LGIS	1	NT SERVICE\SQLSERVERAGENT	NT Service\SQLSERVERAGENT		-- network protocol: LPC set quoted_identifier on set...	SQLAgent - Schedule Saver	local machine
2026-08-05 18:23:37.0069950	LGO	1	sb_customer01	SB_CUSTOMERO1			SQL Server Management Studio	local machine

Figure 13. SQL Server login audit including failed and successful customer logins.

Successful login, logout and failed-login (LGIF) events for SB_CUSTOMERO1 are summarized in figure 13. Failed authentication thus does not go away in the back of the database evidence trail, but rather appears to the Security Auditor. This helps to detect password guessing, configuration problems and unauthorised access attempts.

6.10 Temporal History and Concurrency Controls

bank.Customer and bank.All the accounts are set up as system-versioned temporal tables with history.CustomerHistory and history.AccountHistory. Thus, SQL Server keeps previous row versions when the protected customer/account data changes, giving an extra source of data from which to investigate. rowversion columns are created on Staff, StaffPrivate, Customer and Account to provide concurrency tokens; CHECK and foreign-key constraints are used to ensure domain and relationship integrity. These controls are complimentary to the audit tables, and are not intended to replace them, as temporal history can answer the question of what the previous row looked like, while audit can answer the questions of who did what, where and how.

7. Availability, Backup and Recovery

Availability is based on a scheduled verified and encrypted backup strategy using FULL recovery model. The adopted Recovery Point Objective (RPO) is 15 minutes since the transaction log backups are performed at 15-minute intervals; during normal operation, the intended maximum period of committed data exposure between successful log backups is about 15 minutes. Daily full backups and six-hour differential backups minimize restore effort, cross-instance restore and PITR ensure that the backup chain is usable and not just there, and checksum verification ensures data integrity. The assignment does not prescribe an RTO, so the report does not claim an untested recovery-time SLA (Microsoft, 2025b).

7.1 SQL Server/Agent status and automated backup jobs

	servicename	status_desc	startup_type_desc	last_startup_time	service_account
1	SQL Server (MSSQLSERVER)	Running	Automatic	2026-08-02 20:52:29.6917023 +08:00	NT Service\MSSQLSERVER
2	SQL Server Agent (MSSQLSERVER)	Running	Automatic	NULL	NT Service\SQLSERVERAGENT

	JobName	enabled	LastOutcome	LastRunDateTime	LastMessage
1	SmartBank - Differential Backup	1	Succeeded	2026-08-04 13:00:07.000	The job succeeded. The Job was invoked by Sched...
2	SmartBank - Full Backup	1	Succeeded	2026-08-04 01:00:00.000	The job succeeded. The Job was invoked by Sched...
3	SmartBank - Transaction Log Backup	1	Succeeded	2026-08-04 16:45:00.000	The job succeeded. The Job was invoked by Sched...

	BackupDate	FullBackups	DifferentialBackups	LogBackups	AES256Backups	DailyReadiness
1	2026-08-02	1	2	55	58	PASS
2	2026-08-03	1	2	49	52	PASS
3	2026-08-04	1	1	32	34	PASS

Figure 14. SQL Server services, scheduled backup jobs and three-day readiness evidence.

Figure 14 shows the SQL Server and SQL Server Agent services running with automatic startup. The enabled jobs are SmartBank - Full Backup, SmartBank - Differential Backup and SmartBank - Transaction Log Backup, with latest outcomes of Succeeded. The daily readiness query reports PASS for 2, 3 and 4 August 2026, proving at least three consecutive days of successful scheduled backup operation. The implemented schedule is a daily full backup at 01:00, differential backup every six hours from 07:00 and transaction-log backup every 15 minutes.

SQL Server Agent provides the recurring execution mechanism for the backup jobs. During the demonstration, job history and audit.BackupEvidence should be shown together so that schedule success, backup-file creation, encryption and RESTORE VERIFYONLY verification are all visible (Microsoft, 2025c).

	BackupEvidenceID	BackupStartUtc	BackupFinishUtc	BackupType	BackupFile	Encrypted	VerifyOnlyPassed	Success	ErrorMessage	SqlAgentJobName
1	20669	2026-08-04 08:45:00.829	2026-08-04 08:45:00.927	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	NULL	SmartBank - Transaction Log Backup
2	20668	2026-08-04 08:30:01.606	2026-08-04 08:30:01.692	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	NULL	SmartBank - Transaction Log Backup
3	20667	2026-08-04 08:15:01.235	2026-08-04 08:15:01.330	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	NULL	SmartBank - Transaction Log Backup
4	20666	2026-08-04 08:00:00.975	2026-08-04 08:00:01.057	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	NULL	SmartBank - Transaction Log Backup
5	20665	2026-08-04 07:45:01.444	2026-08-04 07:45:01.514	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	NULL	SmartBank - Transaction Log Backup
6	20664	2026-08-04 07:30:01.641	2026-08-04 07:30:01.730	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	NULL	SmartBank - Transaction Log Backup
7	20663	2026-08-04 07:15:00.888	2026-08-04 07:15:00.971	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	NULL	SmartBank - Transaction Log Backup
8	20662	2026-08-04 07:00:01.068	2026-08-04 07:00:01.164	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	NULL	SmartBank - Transaction Log Backup
9	20661	2026-08-04 06:45:01.344	2026-08-04 06:45:01.420	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	NULL	SmartBank - Transaction Log Backup

Figure 15. Detailed transaction-log backup evidence records.

Figure 15 shows repeated transaction-log backup records in audit.BackupEvidence, including backup timestamps, physical file names, Encrypted = 1, VerifyOnlyPassed = 1, Success = 1 and the SQL Agent job name. The absence of error messages, together with repeated scheduled executions, provides auditable evidence that the log-backup component of the 15-minute RPO has been functioning.

backup_set_id	database_name	Backup Type	backup_start_date	backup_finish_date	is_copy_only	has_backup_checksums	is_compressed	is_encrypted	key_algorithm	encryptor_type	BackupSizeMB	CompressedBackupSizeMB	physical_device_name
172	2495	SmartBankDB	LOG	2026-08-01 17:15:00.000	2026-08-01 17:15:00.000	0	1	1	aes_256	CERTIFICATE_OAEP_256	0.14	0.02	C:\Program Files\Micros
173	2494	SmartBankDB	LOG	2026-08-01 17:00:01.000	2026-08-01 17:00:01.000	0	1	1	aes_256	CERTIFICATE_OAEP_256	0.14	0.02	C:\Program Files\Micros
174	2493	SmartBankDB	LOG	2026-08-01 16:45:01.000	2026-08-01 16:45:01.000	0	1	1	aes_256	CERTIFICATE_OAEP_256	0.14	0.02	C:\Program Files\Micros
175	2492	SmartBankDB	LOG	2026-08-01 16:30:00.000	2026-08-01 16:30:00.000	0	1	1	aes_256	CERTIFICATE_OAEP_256	0.14	0.02	C:\Program Files\Micros
176	2491	SmartBankDB	LOG	2026-08-01 16:15:01.000	2026-08-01 16:15:01.000	0	1	1	aes_256	CERTIFICATE_OAEP_256	0.14	0.02	C:\Program Files\Micros
177	2490	SmartBankDB	LOG	2026-08-01 16:00:01.000	2026-08-01 16:00:01.000	0	1	1	aes_256	CERTIFICATE_OAEP_256	0.14	0.02	C:\Program Files\Micros
178	2489	SmartBankDB	LOG	2026-08-01 15:45:00.000	2026-08-01 15:45:00.000	0	1	1	aes_256	CERTIFICATE_OAEP_256	0.08	0.01	C:\Program Files\Micros
179	2488	SmartBankDB	LOG	2026-08-01 15:30:01.000	2026-08-01 15:30:01.000	0	1	1	aes_256	CERTIFICATE_OAEP_256	0.21	0.02	C:\Program Files\Micros
180	2487	SmartBankDB	LOG	2026-08-01 15:15:01.000	2026-08-01 15:15:01.000	0	1	1	aes_256	CERTIFICATE_OAEP_256	0.14	0.02	C:\Program Files\Micros

Figure 16. SQL Server backup metadata confirms AES-256 encryption.

Figure 16 independently confirms the backup properties through msdb metadata: has_backup_checksums = 1, is_compressed = 1, is_encrypted = 1, key_algorithm = AES_256 and certificate-based encryption. This is stronger than simply showing a .bak/.trn file because SQL Server itself records the media-set encryption and verification characteristics.

SQL Server backup encryption supports AES algorithms with a certificate or asymmetric key. SmartBank uses AES-256 and exports the backup certificate/private key required for disaster recovery. The certificate and private key must be protected separately because the encrypted backup cannot be restored on another instance without the corresponding key material (Microsoft, 2026f).

7.2 Cross-instance encrypted restore preparation

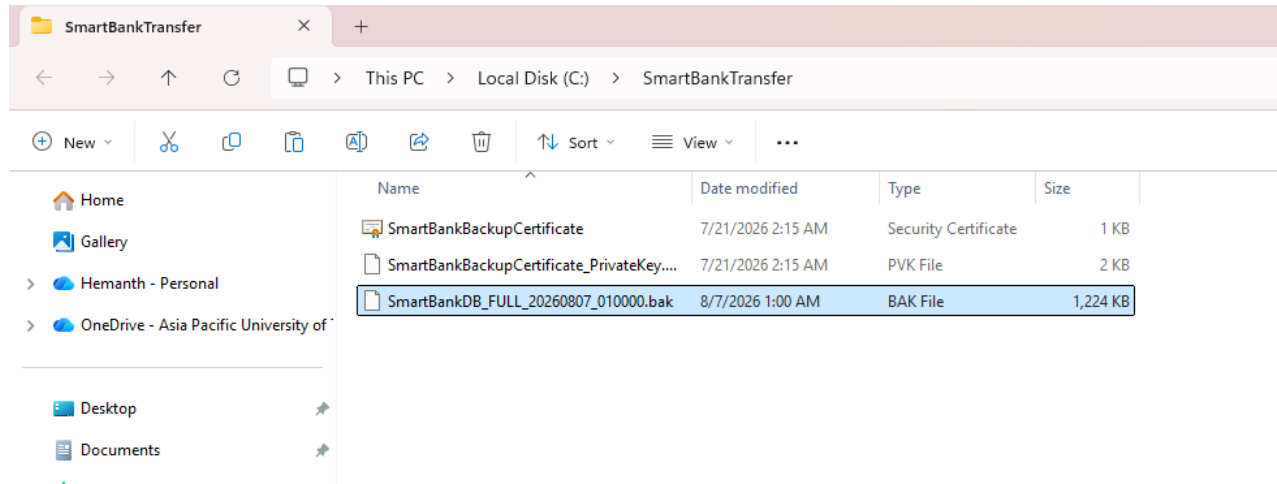


Figure 17. Transferred encrypted backup, certificate and private key files.

The C:\SmartBankTransfer folder contains the full .bak file, the backup certificate and the corresponding private key file. The following are the artefacts that are required to restore a SQL Server backup that was encrypted on another instance.

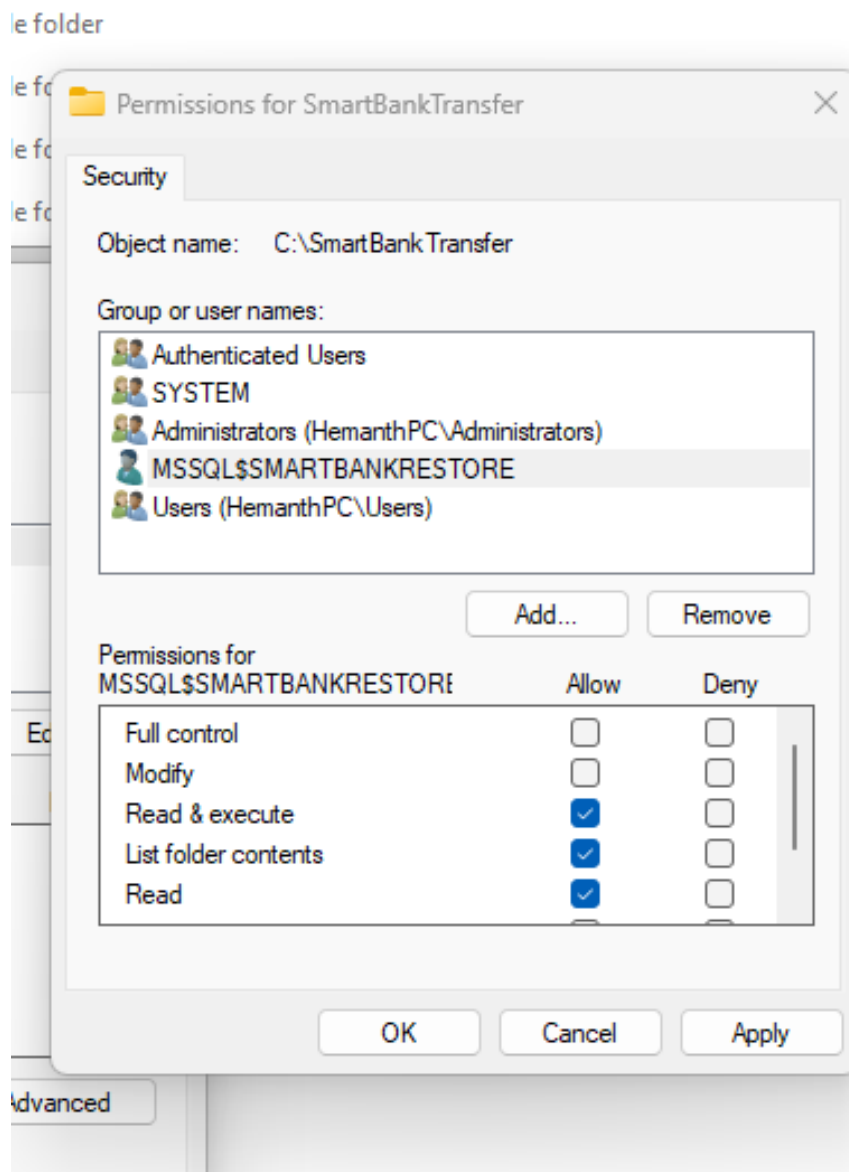


Figure 18. Destination SQL Server service account has read access to transfer folder.

MSSQL\$SMARTBANKRESTORE has Read & execute, List folder contents and Read permissions on C:\SmartBankTransfer in the Windows permissions dialog. This will make sure that the encrypted backup and key/certificate files will be able to be decrypted on the destination SQL Server service without unnecessary write/full-control permissions.

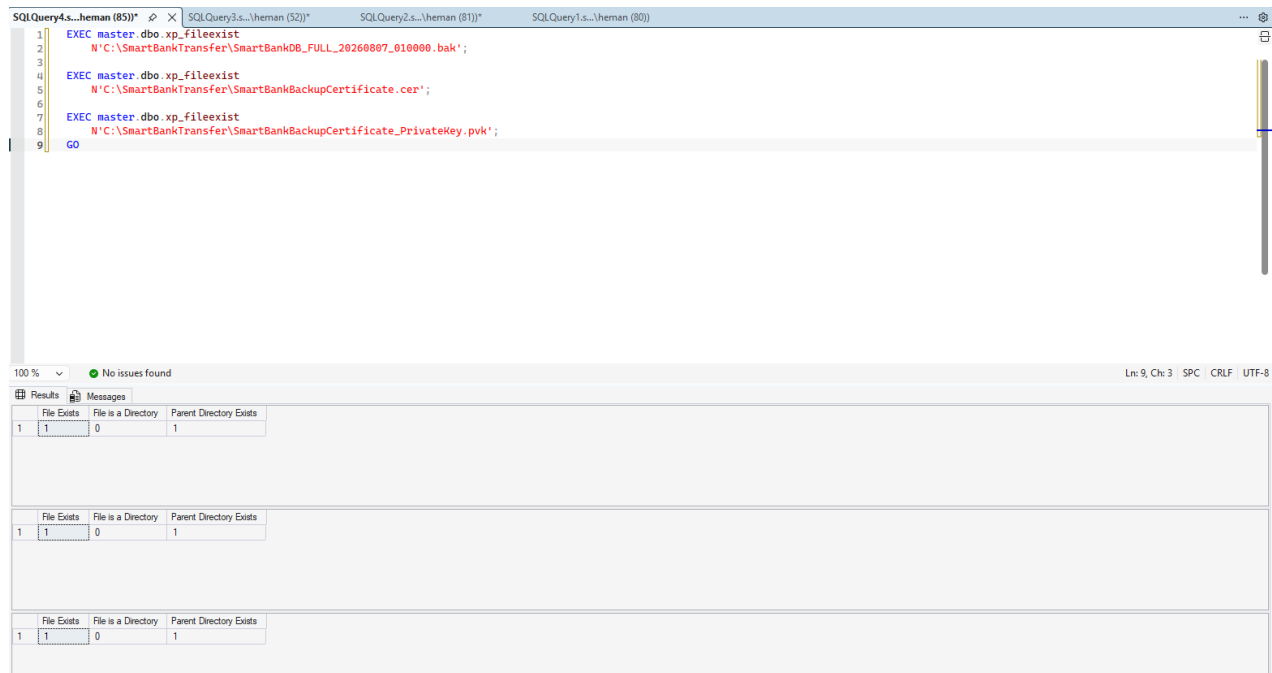


Figure 19. SQL Server confirms all transferred restore files exist.

xp_fileexist is run for the .bak, .cer and .pvk files, returning File Exists = 1 and a valid parent directory for each of these files. This ensures that the file is visible in the SQL Server context before trying to restore.

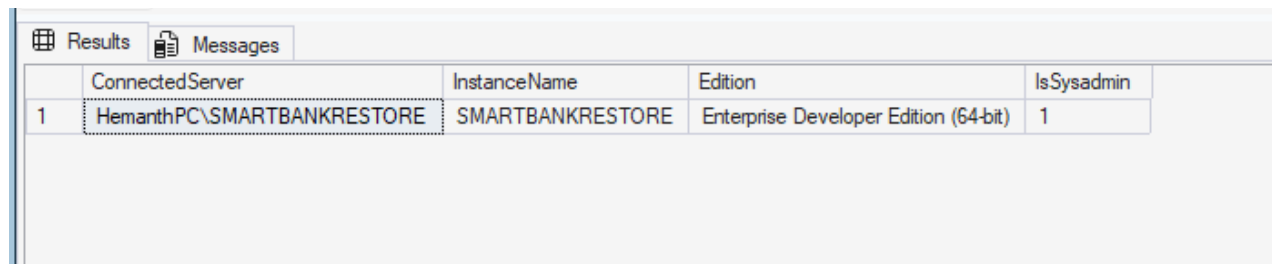


Figure 20. Separate destination SQL Server instance identified.

The destination is HemanthPC\SMARTBANKRESTORE; SERVERPROPERTY confirms the named SMARTBANKRESTORE instance and a sysadmin administrative session. This provides explicit evidence that the encrypted restore is performed on a different SQL Server instance from the source MSSQLSERVER database engine.

7.3 Encrypted backup restored on the second SQL Server instance

```

1  /*
2  SmartBankDB - Restore an AES-256 encrypted backup on another SQL Server instance
3
4  SOURCE INSTANCE REQUIREMENTS
5  - Run 08_Encrypted_Backup_Automation_and_Readiness.sql first.
6  - Copy these three files to a folder readable by the DESTINATION SQL Server service:
7    1. A SmartBankDB_FULL*.bak file
8    2. SmartBankBackupCertificate.cer
9    3. SmartBankBackupCertificate_Privatekey.pvk
10
11 DESTINATION INSTANCE REQUIREMENTS
12 - Run this script as sysadmin on the second SQL Server instance.
13 - Change @TransferFolder and @FullBackupFile before executing.
14 - The script restores as SmartBankDB_Restored so the active database is untouched.
15 */
16
17 USE master;
18
19 SET NOCOUNT ON;
20
21 GO
22
23 /* ===== EDIT THESE VALUES ===== */
24 DECLARE @TransferFolder nvarchar(4000) = N'C:\SmartBank\transfer\';
25 DECLARE @FullBackupFile nvarchar(4000) = N'C:\SmartBank\transfer\SmartBankDB_FULL_20260807_010000.bak';
26 DECLARE @RestoreDatabase sysname = N'SmartBankDB_Restored';

```

BackupName	BackupDescription	BackupType	ExpirationDate	Compressed	Position	DeviceType	UserName	ServerName	DatabaseName	DatabaseVersion	DatabaseCreationDate	BackupSize	FirstLSN	LastLSN	CheckpointLSN	DatabaseBackupLSN	BackupStartDate
1	SmartBankDB	1	NULL	1	1	2	NT SERVICE\SQLSERVERAGENT	HEMANTHPC	SmartBankDB	999	2026-07-21 02:15:29.000	11624448	700000000040800001	700000000043200001	700000000040800001	700000000178400001	2026-08-07 01:00:00.000

LogicalName	PhysicalName	Type	FileGroupName	Size	MaxSize	Field	Created_LSN	Dropped_LSN	Uniqued	ReadOnly_LSN	ReadWrite_LSN	BackupSizeInBytes	SourceBlockSize	FileGroupID	LogGroupGUID	DifferentialBaseLSN	DifferentialBaseGUID
1	SmartBankDB	D	PRIMARY	75497472	35184372080640	1	0	0	D509B92B-C7AF-41C0-8B10-7C8E2EF328E	0	0	9830400	4096	1	NULL	700000000178400001	7AB9FE79-7
2	SmartBankDB_Log	L	NULL	8388608	2199023255552	2	0	0	5432B9F8-38CC-45BD-BAED-2A1019FD1666	0	0	0	4096	0	NULL	0	00000000-0

name	state_desc	recovery_model_desc	page_verify_option_desc
SmartBankDB_Restored	ONLINE	FULL	CHECKSUM

Query executed successfully. HEMANTHPC\SMARTBANKRESTORE ... HEMANTHPC\hemant (88) master 00:00:01 Rows: 1, Cols: 1 1 rows

Figure 21. AES-256 encrypted backup restored as SmartBankDB_Restored.

Figure 21 shows the AES-256 encrypted full backup being restored on SMARTBANKRESTORE as SmartBankDB_Restored. Backup/file metadata is inspected, the required backup certificate/private key is imported, and the restored database is reported ONLINE with FULL recovery and CHECKSUM page verification. The restored copy contains the expected customer data and the restore completes without a SQL Server runtime error.

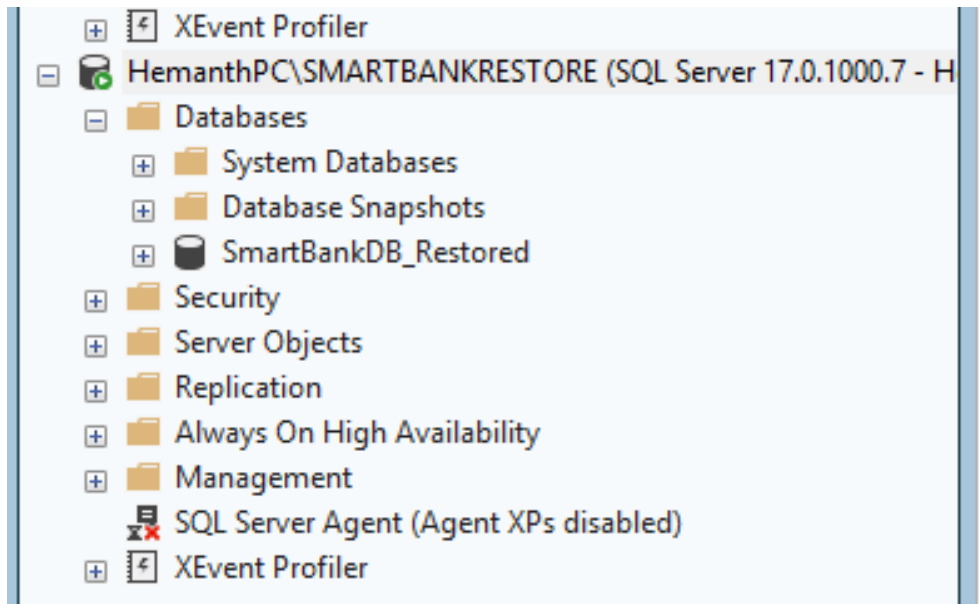


Figure 22. Restored database visible in destination Object Explorer.

On HemanthPC\SMARTBANKRESTORE, you will find SmartBankDB_Restored database under Databases. This gives the user GUI-level assurance the database has been restored on the destination instance.

Results

Messages

LoginName	DatabaseUser	DatabaseName
SB_CUSTOMER01	SB_CUSTOMER01	SmartBankDB_Restored

CustomerID	CustomerName	Email	Phone	DateOfBirth	ICNumber	Address	CustomerStatus	ModifiedAt
CU0001	Nur Izzati Hassan	izzati@example.com	012-3456789	1998-04-12	980412-14-5678	12 Jalan Ampang, 50450 Kuala Lumpur	Active	2026-07-20 18:15:31.143

AccountID	CustomerID	AccountType	Balance	AccountStatus	OpenedAt	ClosedAt	ModifiedAt
AC00000001	CU0001	Savings	5035.00	Active	2026-07-20 18:15:31.156	NULL	2026-08-05 14:45:24.938
AC00000002	CU0001	Current	1500.00	Active	2026-07-20 18:15:31.156	NULL	2026-07-20 18:15:31.156

TransID	TransactionReference	TransferGroupID	CustomerID	AccountID	CounterpartyAccountID	TransDate	Amount	TransactionType	Direction	BalanceBefore	BalanceAfter	
1	7	E9CA2F4A-DC90-F111-ADFD-580205C41827	2A8F4E55-B4C6-4CAC-A4DD-61AB711D416B	CU0001	AC00000001	AC00000003	2026-08-05 14:45:24.940	5.00	Transfer	Debit	5040.00	5035.00
2	6	E8CA2F4A-DC90-F111-ADFD-580205C41827	NULL	CU0001	AC00000001	NULL	2026-08-05 14:45:24.818	10.00	Withdrawal	Debit	5050.00	5040.00
3	5	E7CA2F4A-DC90-F111-ADFD-580205C41827	NULL	CU0001	AC00000001	NULL	2026-08-05 14:45:24.704	50.00	Deposit	Credit	5000.00	5050.00
4	2	3B519DFD-6684-F111-ADFD-580205C41827	NULL	CU0001	AC00000002	NULL	2026-07-11 18:15:31.167	1500.00	Deposit	Credit	0.00	1500.00
5	1	3A519DFD-6684-F111-ADFD-580205C41827	NULL	CU0001	AC00000001	NULL	2026-07-10 18:15:31.167	5000.00	Deposit	Credit	0.00	5000.00

Figure 23. SB_CUSTOMER01 accesses restored data with normal role restrictions.

The result set contains a LoginName SB_CUSTOMER01, DatabaseUser SB_CUSTOMER01 and DatabaseName SmartBankDB_Restored. The customer will be able to get the decrypted CU0001 profile, and can only access the rows of the corresponding account and transactions. This is an example that the recovered database isn't just an online database – its authentication, authorised decryption, and row-restriction behaviour are also usable.

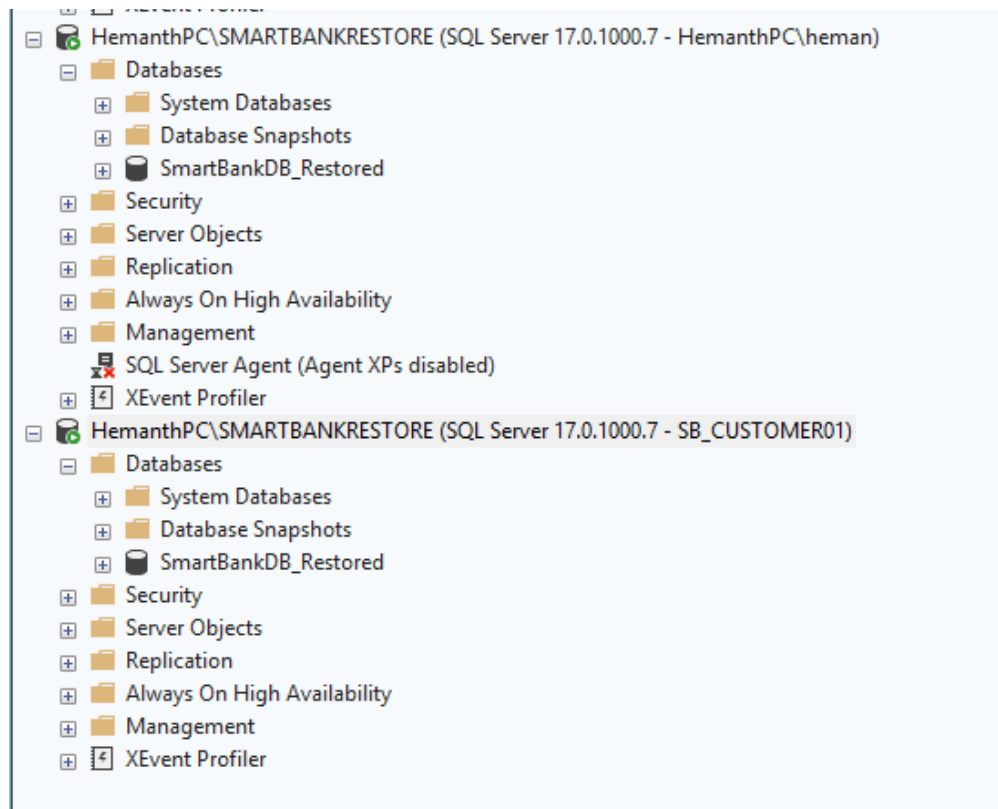
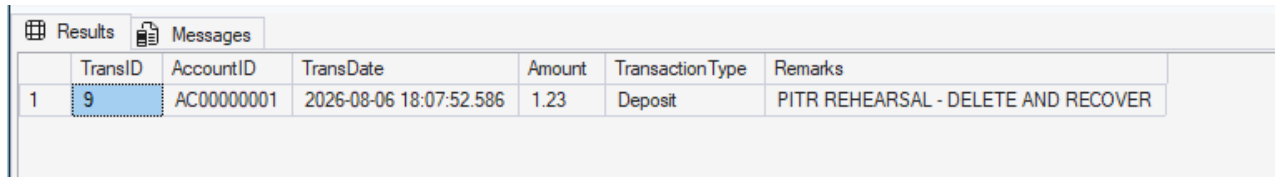


Figure 24. Administrative and customer connections to the restored instance.

There are two connections displayed for HemanthPC\SMARTBANKRESTORE: one Windows connection and one SQL connection (SB_CUSTOMER01). Both may view SmartBankDB_Restored, and, if allowed, will be informed in the GUI that the second instance is functioning normally for authenticated use.

7.4 Point-in-time recovery (PITR) test

This PITR rehearsal intentionally removes TransID 9, takes a snapshot of the transaction-log at the moment it was deleted, copies a separate SmartBankDB_PITR copy to a timestamp just before the deletion, finds the row missing from the PITR copy and restores the row to the active SmartBankDB. In the FULL recovery model, SQL Server will take the full backup and the necessary log backup and then use the STOPAT option to recover to the chosen recovery point (Microsoft, 2024).

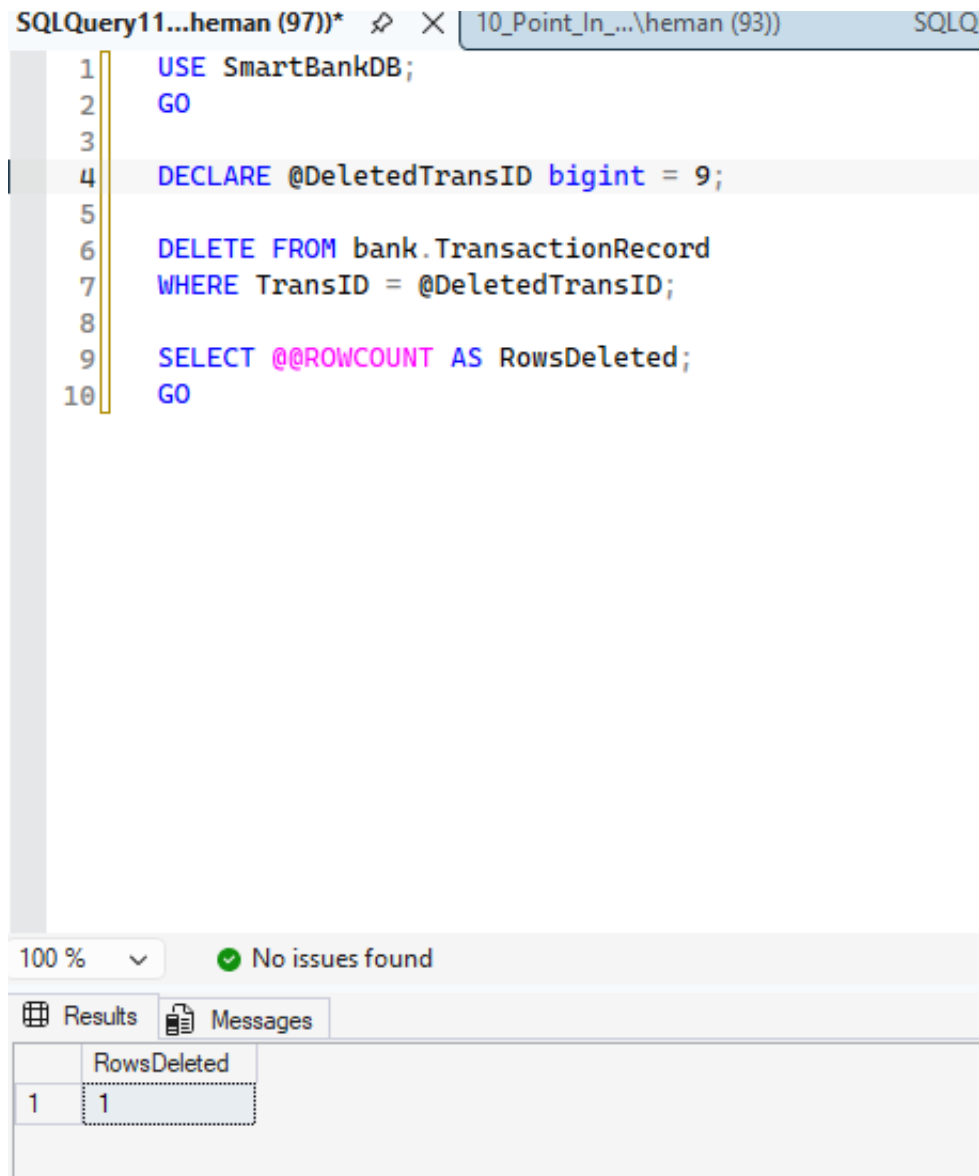


The screenshot shows the 'Results' pane of SQL Server Enterprise Manager. It displays a single row of transaction log data. The 'TransID' column has the value '9' highlighted. The 'Remarks' column contains the text 'PITR REHEARSAL - DELETE AND RECOVER'.

	TransID	AccountID	TransDate	Amount	TransactionType	Remarks
1	9	AC00000001	2026-08-06 18:07:52.586	1.23	Deposit	PITR REHEARSAL - DELETE AND RECOVER

Figure 25. Transaction 9 exists before the recovery test.

Recovery baseline: Figure 25 shows that the TransID 9 is present in the active database prior to the controlled deletion, and that account, timestamp, amount, transaction type and remarks are displayed. This baseline is necessary to later verify the recovery in a clear and unambiguous manner.



```
SQLQuery11...heman (97))* 10_Point_In_...\heman (93)) SQLQ
1  USE SmartBankDB;
2  GO
3
4  DECLARE @DeletedTransID bigint = 9;
5
6  DELETE FROM bank.TransactionRecord
7  WHERE TransID = @DeletedTransID;
8
9  SELECT @@ROWCOUNT AS RowsDeleted;
10 GO
```

100 % No issues found

Results Messages

	RowsDeleted
1	1

Figure 26. Transaction 9 deliberately deleted from the active database.

The DELETE statement targets TransID 9 and the result returns RowsDeleted = 1. This creates a controlled data-loss incident for the PITR demonstration.

```

47
48
49 /* -----
50 PHASE B - AFTER THE DELETION, TAKE AN IMMEDIATE ENCRYPTED LOG BACKUP
51 ----- */
52 EXEC admin.usp_RunBackup
53     @BackupType = 'LOG',
54     @JobName = N'Manual post-deletion PITR log backup';
55 GO

```

100 % 2 0

Results Messages

100 percent processed.
 Processed 21 pages for database 'SmartBankDB', file 'SmartBankDB_log' on file 1.
 BACKUP LOG successfully processed 21 pages in 0.003 seconds (54.687 MB/sec).
 The backup set on file 1 is valid.

Completion time: 2026-08-07T02:16:45.5408515+08:00

Figure 27. Immediate encrypted transaction-log backup taken after deletion.

The script uses `admin.usp_RunBackup`, passing it the value of `LOG` for the `BackupType` parameter and a manual post-deletion PITR job name. The back up of the log succeeds, according to SQL Server. If the incident occurs, the log is captured and the log chain will remain for analysis of the incident.

SQLQuery12...heman (81)* SQLQuery11...heman (97)* 10_Point_In_...heman (93) SQLQuery10...heman (88)* SQLQuery9.s...OMER01 (73)*

```

1 USE SmartBankDB;
2 GO
3
4 SELECT TOP (5)
5     BackupEvidenceID,
6     BackupStartUtc,
7     BackupFinishUtc,
8     BackupType,
9     BackupFile,
10    Encrypted,
11    VerifyOnlyPassed,
12    Success,
13    SqlAgentJobName
14 FROM audit.BackupEvidence
15 ORDER BY BackupEvidenceID DESC;
16 GO

```

100 % No issues found Ln: 16, Ch: 3 SPC CRLF UTF-8

	BackupEvidenceID	BackupStartUtc	BackupFinishUtc	BackupType	BackupFile	Encrypted	VerifyOnlyPassed	Success	SqlAgentJobName
1	20778	2026-08-06 18:16:45.423	2026-08-06 18:16:45.512	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	Manual post-deletion PITR log backup
2	20777	2026-08-06 18:15:01.368	2026-08-06 18:15:01.454	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	SmartBank - Transaction Log Backup
3	20776	2026-08-06 18:09:01.254	2026-08-06 18:09:01.391	FULL	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	PITR rehearsal baseline full backup
4	20775	2026-08-06 18:02:39.599	2026-08-06 18:02:40.077	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	Manual post-deletion PITR log backup
5	20774	2026-08-06 18:00:00.842	2026-08-06 18:00:00.928	LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	1	1	SmartBank - Transaction Log Backup

Figure 28. BackupEvidence confirms the PITR rehearsal backups.

The evidence table displays a manual post-deletion PITR log back up, a PITR rehearsal baseline full backup, and scheduled transaction-log backups. All rows displayed have `Encrypted = 1`, `VerifyOnlyPassed = 1`, `Success = 1`, which is a good sign that the recovery chain is reliable.

Backup Type	Backup File	Position	Backup Finish	First LSN	Last LSN
1 FULL	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	2026-08-07 02:09:01.000	NULL	NULL
2 LOG	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	1	2026-08-07 02:15:01.000	78000000122400001	78000000214400001

Logical Name	Physical Name	Type	File Group Name	Size	Max Size	Field	Create LSN	Drop LSN	Uniqued	Read Only LSN	Read Write LSN
1 SmartBankDB	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	D	PRIMARY	75497472	35184372080640	1	0	0	D509B92B-C7AF-41C0-BB1D-7CEB2EF632BE	0	0
2 SmartBankDB_Log	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	L	NULL	8388608	2199023255552	2	0	0	5432B9F8-38CC-45BD-BAED-2A1019FD1666	0	0

Command Order	Restore Command
1	RESTORE DATABASE [SmartBankDB_PITR] FROM DISK = ...
2	RESTORE LOG [SmartBankDB_PITR] FROM DISK = N'C:\Pr...

Restored Database	Staff Rows	Customer Rows	Account Rows	Transaction Rows
1 SmartBankDB_PITR	4	4	5	9

Figure 29. PITR restore sequence and recovered database row counts.

Figure 29 shows the selected full/log backup inputs, generated RESTORE DATABASE/RESTORE LOG sequence and the resulting SmartBankDB_PITR row counts. The restored copy contains four staff rows, four customer rows, five account rows and nine transaction rows, confirming that the recovery copy is populated at the chosen point in time.

name	state_desc	recovery_model_desc	page_verify_option_desc
1 SmartBankDB	ONLINE	FULL	CHECKSUM
2 SmartBankDB_PITR	ONLINE	FULL	CHECKSUM

Figure 30. SmartBankDB and SmartBankDB_PITR are both online.

The databases are displayed ONLINE with FULL recovery model and CHECKSUM page verification. While the separate PITR copy is utilized to find the deleted record, the active database can be utilised at the same time.

The screenshot shows a SQL Server Enterprise Manager interface. The top pane displays a SQL query with the following code:

```

1 DECLARE @DeletedTransID bigint = 9;
2
3 SELECT
4     N'ACTIVE DATABASE' AS DatabaseCopy,
5     TransID,
6     AccountID,
7     Amount,
8     TransactionType,
9     Remarks
10  FROM SmartBankDB.bank.TransactionRecord
11  WHERE TransID = @DeletedTransID;
12
13 SELECT
14     N'PITR DATABASE' AS DatabaseCopy,
15     TransID,
16     AccountID,
17     Amount,
18     TransactionType,
19     Remarks
20  FROM SmartBankDB_PITR.bank.TransactionRecord
21  WHERE TransID = @DeletedTransID;
22  GO
  
```

The bottom pane shows the query results. The "Results" tab is active, displaying a grid with the following data:

	DatabaseCopy	TransID	AccountID	Amount	TransactionType	Remarks
1	PITR DATABASE	9	AC00000001	1.23	Deposit	PITR REHEARSAL - DELETE AND RECOVER

Figure 31. Deleted transaction absent from active database but present in PITR copy.

The comparison query does not return any row for the active database, but the PITR database returns the original transaction. This is the most important evidence that the point-in-time restore reached a state prior to the deletion.

```

1 DECLARE @RecoveredTransID bigint = 9;
2
3 SELECT N'ACTIVE' AS DatabaseCopy, *
4 FROM SmartBankDB.bank.TransactionRecord
5 WHERE TransID = @RecoveredTransID;
6
7 SELECT N'PITR' AS DatabaseCopy, *
8 FROM SmartBankDB_PITR.bank.TransactionRecord
9 WHERE TransID = @RecoveredTransID;
10 GO

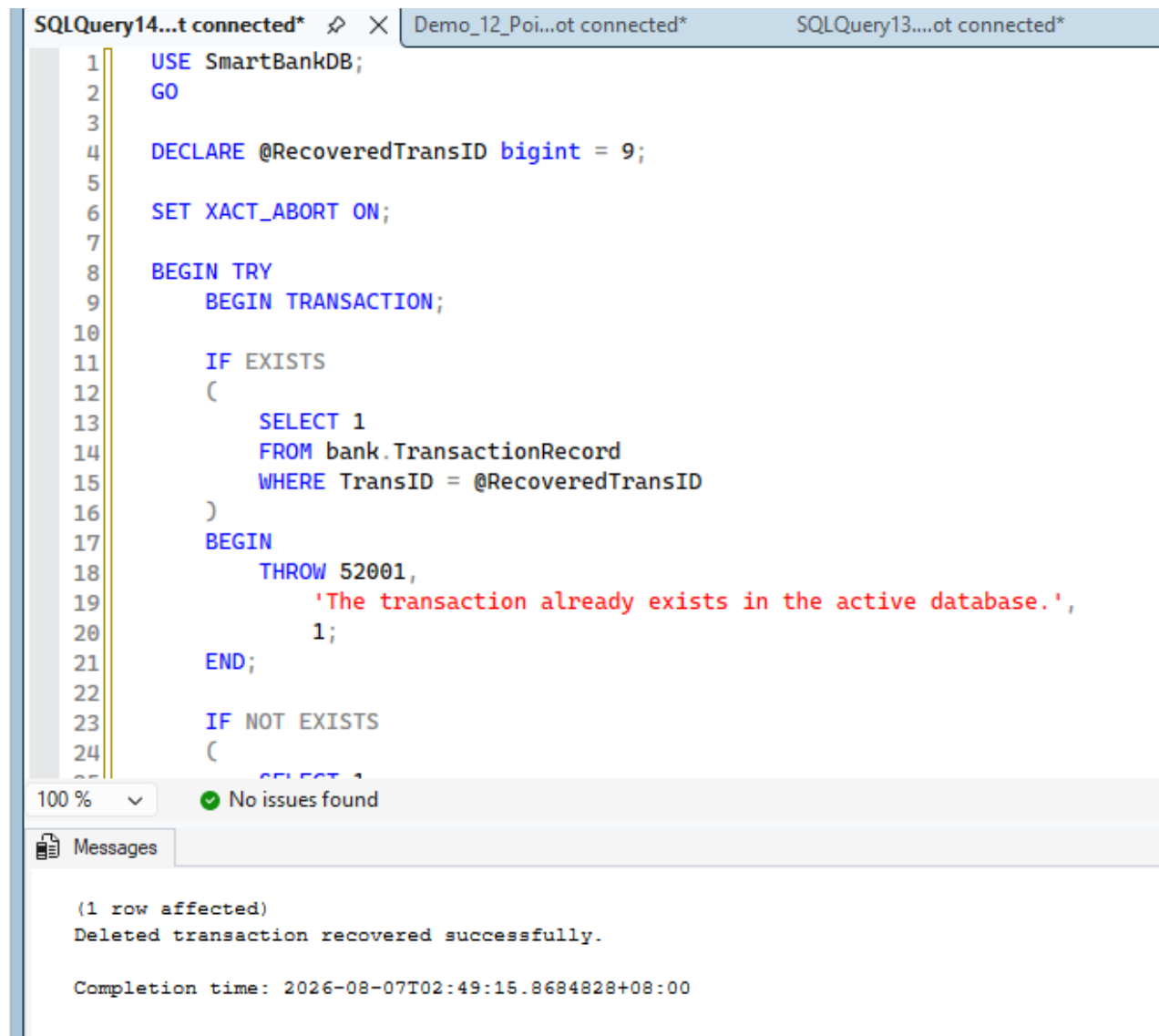
```

100 % No issues found Ln: 1, Ch: 37 TABS CRLF UTF-8

	DatabaseCopy	TransID	TransactionReference	TransferGroupID	CustomerID	AccountID	CounterpartyAccountID	TransDate	Amount	TransactionType	Direction	BalanceBefore	BalanceAfter	Recovery
1	ACTIVE	9	77254DBD-C191-F111-ADFD-580205C41827	NULL	CU0001	AC00000001	NULL	2026-08-06 18:07:52.586	1.23	Deposit	Credit	5035.00	5036.23	PITR
1	PITR	9	77254DBD-C191-F111-ADFD-580205C41827	NULL	CU0001	AC00000001	NULL	2026-08-06 18:07:52.586	1.23	Deposit	Credit	5035.00	5036.23	PITR

Figure 32. Post-recovery comparison of active and PITR copies for TransID 9.

Figure 32 is a post-recovery comparison. TransID 9 is present in both the active SmartBankDB and SmartBankDB_PITR copies, confirming that the missing row has been returned to the active database while the PITR copy remains available as recovery evidence.



The screenshot displays the SQL Server Enterprise Manager interface. At the top, there are three tabs: 'SQLQuery14...t connected*', 'Demo_12_Poi...ot connected*', and 'SQLQuery13....ot connected*'. The active tab, 'SQLQuery14...t connected*', shows a T-SQL script with line numbers 1 through 25. The script is as follows:

```
1  USE SmartBankDB;
2  GO
3
4  DECLARE @RecoveredTransID bigint = 9;
5
6  SET XACT_ABORT ON;
7
8  BEGIN TRY
9      BEGIN TRANSACTION;
10
11     IF EXISTS
12     (
13         SELECT 1
14         FROM bank.TransactionRecord
15         WHERE TransID = @RecoveredTransID
16     )
17     BEGIN
18         THROW 52001,
19             'The transaction already exists in the active database.',
20             1;
21     END;
22
23     IF NOT EXISTS
24     (
25         SELECT 1
```

Below the script, a status bar indicates '100 %' zoom and 'No issues found' with a green checkmark icon. At the bottom, a 'Messages' pane shows the following output:

```
(1 row affected)
Deleted transaction recovered successfully.

Completion time: 2026-08-07T02:49:15.8684828+08:00
```

Figure 33. Recovered row inserted back into active database.

Controlled recovery operation is shown in figure 33. The script first checks to see if the row exists in bank and not in SmartBankDB_PITR, sets the IDENTITY_INSERT option to 'ON' for the recovery insert, and starts an explicit transaction. The script first checks that the row is not in the active bank but is in SmartBankDB_PITR, sets the IDENTITY_INSERT option to 'ON' for the recovery insert and starts an explicit transaction, and commits the original TransID and transaction values back into bank.TransactionRecord. SQL Server reports one row affected, and 'Deleted transaction recovered successfully.'

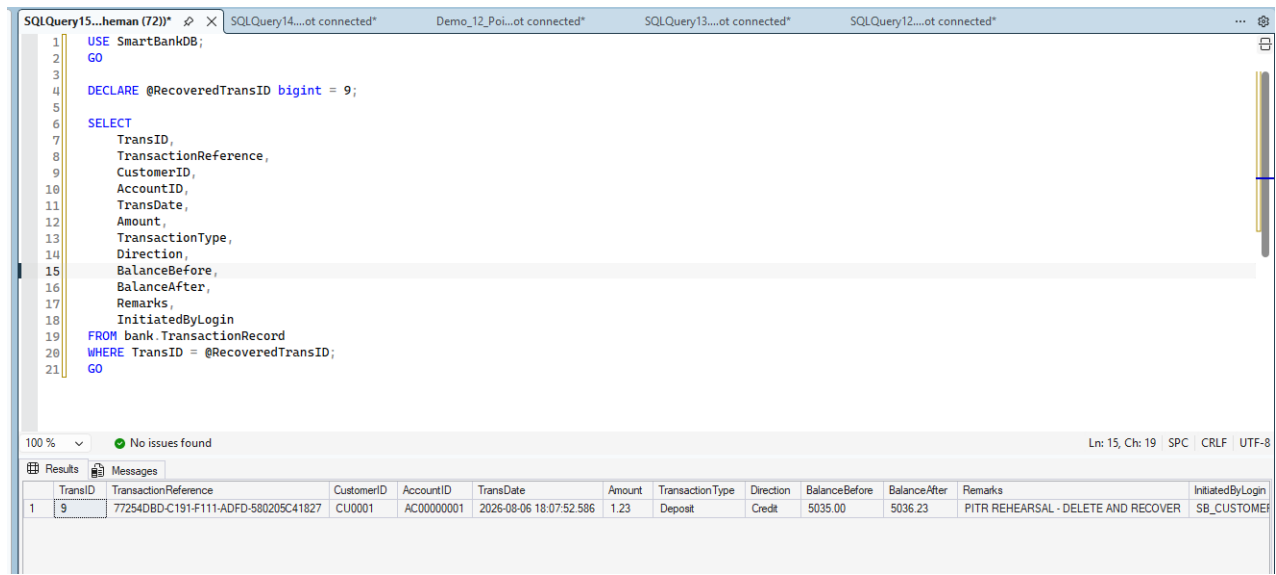


Figure 34. Final verification of recovered transaction 9.

Figure 34 performs the final SELECT from bank.TransactionRecord and returns the same TransID 9 with the original transaction reference, customer/account, timestamp, amount, type, direction, balances, remarks and initiating login. This completes the PITR test by proving that the deleted record is again available in the active database.

8. Consolidated Test Cases

The evidence is summarized by repeatable test cases in the following table. “PASS” indicates that a screen shot shows the desired behavior. These test cases can be re-used in the live demonstration.

ID	Test	Expected result	Result	Evidence
TC-01	Authenticate as different roles	DBA, manager, officer and customer logins map to correct database users/roles	PASS	Figures 2, 4-7
TC-02	DBA confidentiality	DBA sees public staff + own record; unauthorized business access denied	PASS	Figure 4
TC-03	Manager confidentiality	Manager sees officers, accounts and transactions; sensitive customer profile masked	PASS	Figure 5
TC-04	Officer confidentiality	Officer sees own staff record, customer accounts/transactions; direct disallowed query denied	PASS	Figure 6
TC-05	Customer row isolation	Customer sees only own profile, accounts and relevant transactions	PASS	Figures 7-8
TC-06	Authorized decryption	Correct customer receives readable protected profile data	PASS	Figure 10
TC-07	Valid customer transactions	Deposit, withdrawal and transfer execute successfully	PASS	Figure 7

TC -08	Invalid transaction	Insufficient-funds withdrawal rejected with error 51307	PASS	Figure 7
TC -09	RLS / direct DML protection	RLS policies enabled and direct base-table DML denied	PASS	Figures 2-3
TC -10	Audit business actions	Successful and failed actions recorded with actor, action and outcome	PASS	Figure 11
TC -11	Audit security changes	CREATE_USER / ADD_ROLE_MEMBER / GRANT / REVOKE recorded	PASS	Figure 12
TC -12	Audit login failures	Failed customer login attempts are recorded	PASS	Figure 13
TC -13	Automated backup jobs	Full, differential and log jobs enabled and succeeded	PASS	Figure 14
TC -14	Three consecutive days	Daily readiness PASS for 2-4 Aug 2026	PASS	Figure 14
TC -15	Backup encryption	msdb metadata shows AES-256 certificate encryption	PASS	Figure 16
TC -16	Cross-instance restore	Encrypted .bak restored as SmartBankDB_Restored on SMARTBANKRESTORE	PASS	Figures 17-23
TC -17	Restored customer access	SB_CUSTOMER01 can use restored database with appropriate restrictions	PASS	Figure 23

TC-18	PITR deletion recovery	TransID 9 deleted, found in PITR copy, recovered and verified	PASS	Figures 25-34
TC-19	Manager staff management	Manager updates a Bank Officer through approved procedure; direct base-table DML remains denied	PASS	Figure 5
TC-20	Officer customer/account management	Officer creates or updates customer/account records through approved procedures	PASS	Figure 6
TC-21	Staff self-update except salary	Staff can update own permitted details; no salary parameter; StaffPrivate direct update denied	IMPLEMENTED - final screenshot recommended	Demo_14 / api.usp_Staff_UpdateOwnRecord
TC-22	Dynamic/static masking and anonymisation	DDM masks operational fields; reporting snapshot removes/reduces direct identifiers	PASS	Figures 3 and 9
TC-23	DML before/after audit	Successful INSERT/UPDATE/DELETE captured with old/new JSON and actor information	IMPLEMENTED - final screenshot recommended	audit.vw_RecentDMLChanges
TC-24	Temporal history	Customer/Account prior versions retained in system-versioned history tables	IMPLEMENTED	SQL 02 + history schema
TC-25	Data dictionary and classification	Queryable dictionary documents classification and protection controls; SQL sensitivity labels enabled	PASS	Section 4.2 + Figure 3
TC-26	Idempotency protection	Repeated financial request with same idempotency key cannot create duplicate posting	IMPLEMENTED	Unique IdempotencyKey index + customer procedures

9. Assessment Rubric Coverage Matrix

The following matrix maps every supplied marking category to the implemented control and the report evidence. The mark values total 100 and are included to make coverage explicit for both submission review and live defence.

Marking area	Marks	Report evidence	Implemented proof
Overview	4	Executive Summary; Sections 1-3	CIA objectives, weaknesses and overall architecture explicitly defined.
Data Dictionary incl. classification	6	Section 4.2; Figure 3	sec.DataDictionary + SQL sensitivity labels.
Authentication and user management	5	Sections 5.1, 6.9	SQL logins/users, password policy, identity mapping and login audit.
Role Based Access Control	10	Sections 5.1-5.2; Figures 2-8	Roles, GRANT/DENY, authorisation matrix, positive/negative role tests.
Row Level Security	10	Section 5.3; Figures 3, 7, 8	Enabled FILTER/BLOCK policies and customer isolation.
View	5	Section 5.4	Security views expose only role-appropriate columns/rows.
Encryption	10	Sections 5.10, 5.12; Figures 9-10	AES-256 column encryption + authorised decryption.
Data Masking - static/dynamic	10	Section 5.11; Figures 3, 9	DDM + reporting.CustomerAnalyticsMasked snapshot.
Hashing and Anonymisation	5	Section 5.11; Figure 9	Salted SHA2-256 + pseudonymous/aggregated analytics.
Backup, schedule, restore/recovery	10	Section 7; Figures 14-34	Automated full/diff/log, 3-day proof, encrypted restore and PITR.
Audit Strategy and Audit Matrix	5	Section 6.5	audit.AuditMatrix + layered audit design.
Login/Logout Audit	5	Section 6.9; Figure 13	Successful login/logout and failed login evidence.
DML Audit	5	Section 6.6	DML triggers + before/after JSON; final PITR audit screenshot recommended.
DDL Audit	5	Section 6.7; Figure 11	DDL trigger + native SQL Server Audit.
Permission Change Audit	5	Section 6.8; Figure 12	CREATE USER/role membership/GRANT/REVOKE evidence.

10. Demonstration Readiness Mapping

Demo requirement	Evidence / suggested live action
1. Login as several users	Use SB_DBA01, SB_MANAGER01, SB_OFFICER01, SB_CUSTOMER01/02. Figures 2 and 4-8 prove the prepared accounts.
2. Show users and permissions for each role	Run the role/user/permission query represented in Figure 2 and the access matrix in Figure 3.
3. Prove permission settings one query at a time	Repeat the DBA/Manager/Officer/Customer tests from Figures 4-8 and show both successful output and error 229 where access is denied.
4. Show audit records for actions in steps 1 and 3	Show business-action, DML, DDL, permission-change and native SQL Server Audit evidence. Use Figures 11-13 and query audit.vw_RecentDMLChanges after the PITR recovery.
5. Show at least 3 days of successful scheduled backups	Show Figure 14-style daily readiness query with PASS for three consecutive days.
6. Restore encrypted backup on another SQL instance	Use SMARTBANKRESTORE, import certificate/private key and restore .bak; then show ONLINE database and customer access (Figures 17-23).
7. Point-in-time restore after lecturer deletes 1-2 records	Record the row, capture the pre-deletion target time, delete 1-2 leaf transaction rows, take a log backup, restore SmartBankDB_PITR with STOPAT, compare active/PITR copies, recover the rows, then show DELETE/INSERT audit evidence (Figures 25-34 + Section 6.6).
8. Add users/change role permissions and prove audit capture	Create or alter a demo user/role membership, then query DDL audit as in Figure 12.
9. Automatic decryption for correct user	Login as SB_CUSTOMER01 and call the approved own-profile procedure, showing readable IC/address while base encrypted columns remain protected (Figure 10).

11. Limitations and Recommended Improvements

The implemented controls satisfy the assessed SmartBank requirements and include several additional defence-in-depth features. The following items are recommended for a production banking environment and are intentionally presented as future improvements rather than controls that are already implemented:

- Store the backup-encryption certificate and private key in secure off-host/off-site custody with documented recovery access. Loss of the required key material would make encrypted backups unrecoverable.
- Protect private-key export files with least-privilege NTFS permissions, controlled transfer procedures and formal recovery-administrator access.
- Schedule recurring restore drills in addition to backup creation. A backup should be considered reliable only when restore and integrity verification are regularly proven.
- Configure SQL Server Agent operators/alerts or external monitoring so that backup, audit and integrity-check failures are reported automatically rather than discovered during a recovery event.
- Periodically reconnect and re-certify access, deactivate dormant accounts and audit and remove members from roles based on current job duties to maintain least privilege.
- Expand current data quality controls, including foreign-key, CHECK, rowversion and temporal. Add formal data quality controls with data quality monitoring, reconciliation alerts and automated exception reporting.
- Forward or archive audit files to secure, controlled storage that has retention and tamper monitoring mechanisms to prevent privileged local users from removing evidence without detection.
- If it is a production bank, consider enterprise key management/HSM integration, high-availability replicas, off-site disaster-recovery copies, immutable backup tiers, privileged-access management and formal disaster-recovery exercises with measured RTO/RPO performance.

12. Conclusion

The final SmartBankDB redesign implements a complete CIA-based database-security architecture in Microsoft SQL Server. Confidentiality is provided by authenticated identities, user-defined roles, explicit DENY/GRANT permissions, secure views, RLS, DDM, sensitivity classification, AES-256 column encryption and controlled authorised decryption. Integrity is provided by role-specific procedures, foreign keys and CHECK constraints, rowversion, idempotency protection, transaction validation, temporal history and layered auditing that covers valid, invalid, DML, DDL, login and permission-change events. Availability is provided by FULL recovery, automated full/differential/log backups, AES-256 encrypted backup media, verification, cross-instance restore and tested PITR.

Most importantly, the evidence shows behavior, not just configuration: unauthorized queries are denied, different customers get different RLS-filtered rows, protected values are decrypted only via authorized own-profile procedure, Managers and Officers can do only what they're supposed to do in terms of administration, invalid withdrawals are rejected and logged, scheduled backups succeed over multiple days, an encrypted backup works on another SQL Server instance, and a deliberately deleted transaction can be recovered from a point in time copy. In addition to the extra auditor, data-dictionary, audit-matrix, temporal, masking and idempotency controls, the solution fulfils the assignment requirements and offers a defensible defense-in-depth implementation.

References

- Asia Pacific University of Technology & Innovation. (2026). CT069-3-3 Database Security Group Assignment: SmartBank Malaysia.
- Asia Pacific University of Technology & Innovation. (2026). CT069-3-3 Database Security lecture materials: Data Protection; Data Encryption; Authentication and Authorization; RBAC; Advanced Permission Management; Auditing; Backup and Recovery.
- Microsoft. (2024). Restore a SQL Server database to a point in time (Full Recovery Model). Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/backup-restore/restore-a-sql-server-database-to-a-point-in-time-full-recovery-model>
- Microsoft. (2025a). Get started with Database Engine permissions. Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/security/authentication-access/getting-started-with-database-engine-permissions>
- Microsoft. (2025b). Back up a transaction log. Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/backup-restore/back-up-a-transaction-log-sql-server>
- Microsoft. (2025c). Configure a SQL Server Agent job schedule. Microsoft Learn. <https://learn.microsoft.com/en-us/ssms/agent/schedule-a-job>
- Microsoft. (2026a). Permissions hierarchy (Database Engine). Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/security/permissions-hierarchy-database-engine>
- Microsoft. (2026b). Row-level security. Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security>
- Microsoft. (2026c). Dynamic data masking. Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/security/dynamic-data-masking>
- Microsoft. (2026d). SQL Data Discovery and Classification. Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/security/sql-data-discovery-and-classification>
- Microsoft. (2026e). SQL Server Audit (Database Engine). Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/security/auditing/sql-server-audit-database-engine>
- Microsoft. (2026f). Backup encryption - SQL Server. Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/backup-restore/backup-encryption>

Appendix A - Evidence Figure Index

Figure	Source screenshot	Evidence purpose
2	demo 01.png	Logins, users, roles, permissions
3	demo 01 .png	Access matrix, RLS, masks, classification
4	demo 02.png	DBA test
5	demo 03.png	Manager test
6	demo 04.png	Officer test
7	demo 05.png	Customer 01 test
8	demo 06.png	Customer 02 test
9	demo 07.png	Encryption/hashing/masked analytics
10	GetMyCustomerProfile.png	Authorized decrypted customer profile
11	demo 08.png	Business/DDL/native audit
12	demo 09.png	User/permission change audit
13	demo 11.png	Login audit
14	1(1).png	SQL Agent and 3-day backup evidence
15	2(1).png	BackupEvidence log records
16	3(1).png	AES-256 backup metadata
17	SmartBankTransferFolder.png	Transfer artefacts
18	permissions for smartbank folder.png	NTFS permissions
19	Confirm SQL Server can see the files(1).png	File visibility
20	new instance name.png	Destination instance
21	Phase 6 — Run the restore.png	Cross-instance restore
22	SmartBankDB_Restored.png	Restored DB in Object Explorer
23	Prove access as a normal customer in restored.png	Customer access, authorised decryption and RLS on restored database

24	Run Demo 13 in restored.png	Administrative and customer connections to restored instance
25	transaction id 9 before deletion.png	PITR baseline row
26	deleting transaction id 9(1).png	Delete transaction 9
27	runing backup after deleting transaction id9.png	Post-deletion log backup
28	confirm backup after deletion of trasnaction id 9(1).png	PITR backup evidence
29	sql 10 phase C.png	PITR restore commands and counts
30	Confirm the PITR database is online(1).png	PITR DB online
31	Proof the deleted row exists in PITR transaction id 09.png	Deleted row found in PITR
32	Compare active and PITR copies(1).png	Post-recovery active/PITR comparison
33	Deleted transaction recovered successfully.(1).png	Recovery operation successful
34	Verify the recovered data.png	Final recovered row verification

Appendix B - SQL Excerpts Visible in Evidence

The full implementation SQL files are submitted separately. The excerpts below correspond to the recovery actions visible in the evidence and are retained here so the lecturer can quickly connect the screenshots to the key T-SQL used during the demonstration.

B.1 Deliberate PITR deletion

```
DECLARE          @DeletedTransID          bigint          =          9;

DELETE           FROM                      bank.TransactionRecord
WHERE            TransID                   =                @DeletedTransID;

SELECT @@ROWCOUNT AS RowsDeleted;
```

B.2 Manual post-deletion log backup

```
EXEC                                                     admin.usp_RunBackup
  @BackupType          =                                'LOG',
  @JobName = N'Manual post-deletion PITR log backup';
```

B.3 Cross-instance restore file visibility check

```
EXEC                                                     master.dbo.xp_fileexist
N'C:\SmartBankTransfer\SmartBankDB_FULL_20260807_010000.bak';
EXEC  master.dbo.xp_fileexist  N'C:\SmartBankTransfer\SmartBankBackupCertificate.cer';
EXEC                                                     master.dbo.xp_fileexist
N'C:\SmartBankTransfer\SmartBankBackupCertificate_PrivateKey.pvk';
```

B.4 Final recovered row verification

```
DECLARE          @RecoveredTransID          bigint          =          9;

SELECT  TransID, TransactionReference, CustomerID, AccountID, TransDate, Amount,
        TransactionType, Direction, BalanceBefore, BalanceAfter, Remarks, InitiatedByLogin
FROM    bank.TransactionRecord
WHERE   TransID = @RecoveredTransID;
```